

เอกสารสรุปโครงงาน

ระบบเช็คชื่อเข้าใช้ห้องสมุด

Library Check-in System

วิทยาลัยเทคนิคนครนายก

เว็บไซต์ที่ใช้งานจริง	ntclibrary.com
ภาษาที่ใช้พัฒนา	PHP 8 · JavaScript · SQL
ฐานข้อมูล	MariaDB — 6 ตาราง 35 คอลัมน์
หน้าเว็บ	9 หน้า
เส้นทาง API	30 เส้นทาง
รูปแบบรายงาน	8 แบบ
จำนวนฟังก์ชัน	235 ฟังก์ชัน
วันที่จัดทำเอกสาร	24 สิงหาคม 2569

เอกสารฉบับนี้จัดทำขึ้นเพื่อใช้ประกอบการนำเสนอโครงงาน

สารบัญ

บทที่ 1 ภาพรวมของโครงการ

- 1.1 ที่มาและปัญหาที่ต้องการแก้
- 1.2 วัตถุประสงค์
- 1.3 ขอบเขตของระบบ
- 1.4 ผู้ใช้งานสองกลุ่ม
- 1.5 ขนาดของระบบโดยสรุป

บทที่ 2 เทคโนโลยีที่ใช้และเหตุผลที่เลือก

- 2.1 เครื่องมือทั้งหมดที่ใช้
- 2.2 สิ่งที่ตั้งใจไม่ใช้ และเหตุผล

บทที่ 3 สถาปัตยกรรมและการแบ่งงานของโค้ด

- 3.1 เส้นทางการทำงานของหนึ่งคำขอ
- 3.2 เหตุผลที่แยกโฟลเดอร์ public กับ src
- 3.3 หน้าที่ของแต่ละไฟล์
- 3.4 เหตุผลที่ใช้ตัวรับคำขอจุดเดียว

บทที่ 4 ฐานข้อมูล

- 4.1 ตารางทั้งหมด 6 ตาราง
- 4.2 ความสัมพันธ์ระหว่างตาราง
- 4.3 แอดมินกับผู้ใช้อยู่ตารางเดียวกันหรือแยกกัน
- 4.4 เหตุผลที่เก็บการเช็คชื่อเป็น “เหตุการณ์”
- 4.5 ดัชนีที่สร้างไว้และเหตุผล

บทที่ 5 ระบบสมาชิกและการแบ่งสิทธิ์

- 5.1 คอลัมน์ที่ใช้แยกสิทธิ์
- 5.2 ตัวกรองสิทธิ์อยู่ตรงไหนในโค้ด

5.3 เหตุผลที่แยกหน้าของแอดมินกับของผู้ใช้

5.4 การกันไม่ให้ระบบเหลือแอดมินศูนย์คน

บทที่ 6 ความปลอดภัย

6.1 การเก็บรหัสผ่าน

6.2 เหตุผลที่ไม่ใช้ MD5 หรือ SHA-1

6.3 การจัดการเมื่อผู้ใช้ลืมห้สผ่าน

6.4 การป้องกัน SQL Injection

6.5 การป้องกัน XSS

6.6 การจัดการเซสชัน

6.7 การป้องกันการเดารหัสผ่าน

6.8 ส่วนหัวความปลอดภัยของ HTTP

6.9 ไฟล์ .env เก็บอะไร

6.10 การวางไฟล์ไว้นอกพื้นที่ที่เว็บเข้าถึงได้

6.11 ข้อจำกัดที่ยังเหลืออยู่

บทที่ 7 พีเจอรและหน้าเว็บทั้งหมด

7.1 หน้าของนักศึกษา

7.2 หน้าของเจ้าหน้าที่

7.3 กลไกอัตโนมัติที่ทำงานเบื้องหลัง

บทที่ 8 ระบบรายงาน

8.1 รายงานทั้ง 8 แบบ

8.2 ตัวกรองร่วมของทุกรายงาน

8.3 การส่งออกไฟล์ และวิธีทำกราฟลงใน PDF

บทที่ 9 การนำระบบขึ้นใช้งานจริง

9.1 Hostinger และโครงไฟล์บนเซิร์ฟเวอร์

9.2 Git และ GitHub ใช้ทำอะไรบ้าง

--

--

9.3 ขั้นตอนการนำขึ้นอัตโนมัติ

9.4 เหตุผลที่ต้องซ่อมก่อนทุกครั้ง

บทที่ 10 การทดสอบระบบ

บทที่ 11 แนวคำถาม-คำตอบสำหรับการนำเสนอ

ภาคผนวก ก อภิธานศัพท์

บทที่ 1 ภาพรวมของโครงการ

1.1 ที่มาและปัญหาที่ต้องการแก้

เดิมห้องสมุดวิทยาลัยเทคนิคนครนายกบันทึกการเข้าใช้บริการด้วยสมุดลงชื่อ วิธีนี้มีปัญหาที่ตามมาหลายอย่าง คือลายมืออ่านยากจนบางครั้งระบุตัวผู้เข้าใช้ไม่ได้ การนับยอดต้องนับด้วยมือที่ละบรรทัดเมื่อต้องทำรายงานประจำเดือน ข้อมูลที่ได้จึงช้าและมีโอกาสคลาดเคลื่อน อีกทั้งไม่สามารถตอบคำถามเชิงบริหารได้ เช่น ช่วงเวลาใดของวันที่มีผู้ใช้มากที่สุด หรือแผนกวิชาใดใช้บริการห้องสมุดน้อยจนควรจัดกิจกรรมส่งเสริม

โครงการนี้จะพัฒนาระบบเว็บสำหรับบันทึกเวลาเข้า-ออกห้องสมุดแบบดิจิทัล โดยให้นักศึกษาเช็คอินด้วยตนเองผ่านโทรศัพท์ และให้เจ้าหน้าที่เห็นข้อมูลสรุปพร้อมออกรายงานได้ทันที

1.2 วัตถุประสงค์

1. เพื่อบันทึกเวลาเข้า-ออกห้องสมุดของนักศึกษาให้ถูกต้องและตรวจสอบย้อนหลังได้
2. เพื่อให้เจ้าหน้าที่ทราบจำนวนผู้ใช้บริการและผู้ที่อยู่ในห้องสมุดขณะนั้นได้ทันที
3. เพื่อจัดทำรายงานสถิติการใช้บริการในรูปแบบไฟล์ที่นำไปใช้ได้ ทั้ง PDF, CSV และ Excel
4. เพื่อให้นักศึกษาตรวจสอบประวัติการเข้าใช้ของตนเองได้ด้วยตนเอง

1.3 ขอบเขตของระบบ

หัวข้อ	ขอบเขต
ผู้ใช้งาน	นักศึกษาและเจ้าหน้าที่ห้องสมุดของวิทยาลัย เทคนิคนครนายก
อุปกรณ์	ใช้งานผ่านเว็บเบราว์เซอร์ ทั้งโทรศัพท์มือถือ และคอมพิวเตอร์
สิ่งที่ระบบบันทึก	เวลาเข้า เวลาออก เวลาที่ตั้งใจจะออก และ ที่มาของการบันทึกออก
วันเปิดให้บริการ	วันจันทร์ถึงวันศุกร์ (ปรับได้จากไฟล์ตั้งค่า)
สิ่งที่ยอยู่นอกขอบเขต	การยืม-คืนหนังสือ การจองที่นั่ง และการ ชำระค่าปรับ

1.4 ผู้ใช้งานสองกลุ่ม

กลุ่ม	ทำอะไรได้	ทำอะไรไม่ได้
นักศึกษา role = student	สมัครสมาชิก เข้าสู่ระบบ เช็คอิน-เช็คเอาท์ ขอเวลา เพิ่ม ดูประวัติการเข้าใช้ของ ตนเอง แก้ไขข้อมูลส่วนตัว และเปลี่ยนรหัสผ่าน	ดูข้อมูลของนักศึกษาคนอื่น เข้าหน้าเจ้าหน้าที่ และเปิด รายงานทุกชนิด
เจ้าหน้าที่ role = admin	ดูภาพรวมการใช้บริการ จัดการทำเนียบสมาชิก บังคับเช็คเอาท์ รีเซตรหัส ผ่านให้นักศึกษา เขียน ประกาศ และออกรายงาน ทั้ง 8 แบบ	ดูรหัสผ่านของผู้ใช้คนใด ก็ตาม (ระบบไม่ได้เก็บไว้ใน รูปแบบที่อ่านได้)

1.5 ขนาดของระบบโดยสรุป

รายการ	จำนวน	รายละเอียด
ตารางในฐานข้อมูล	6	students, users, checkin_logs, login_attempts, app_settings, schema_migrations
คอลัมน์รวม	35	นับรวมทั้ง 6 ตาราง
หน้าเว็บ	9	หน้าของนักศึกษา 4 หน้า หน้าเจ้าหน้าที่ 4 หน้า และหน้าเลือกรายงาน 1 หน้า
เส้นทาง API	30	ทุกเส้นทางผ่านไฟล์ index.php ไฟล์เดียว
รูปแบบรายงาน	8	ส่งออกได้เป็น PDF, CSV และ Excel
ฟังก์ชันในโค้ด	235	ฝั่งเซิร์ฟเวอร์ (PHP) และฝั่งเบราว์เซอร์ (JavaScript)
ไลบรารีภายนอก	2	mPDF สำหรับสร้าง PDF และ PhpSpreadsheet สำหรับอ่านไฟล์รายชื่อ

บทที่ 2 เทคโนโลยีที่ใช้และเหตุผลที่เลือก

2.1 เครื่องมือทั้งหมดที่ใช้

เครื่องมือ	ใช้ทำอะไร	เหตุผลที่เลือกสิ่งนี้
PHP 8	ภาษาหลักฝั่งเซิร์ฟเวอร์ ทั้งการรับคำขอ ติดต่อ ฐานข้อมูล และสร้างหน้า เว็บ	โฮสต์ราคาประหยัดของไทยแทบทุก เจ้ารองรับ PHP โดยไม่ต้องตั้งค่า อะไรเพิ่ม ต่างจากภาษาอื่นที่ต้องใช้ เซิร์ฟเวอร์เฉพาะ
MariaDB	ฐานข้อมูลเก็บข้อมูล ทั้งหมด	มาพร้อมกับแพ็คเกจโฮสต์ รองรับ SQL มาตรฐาน และมีฟังก์ชัน หน้าต่าง (window function) ที่ระบบ ใช้จับคู่เวลาเข้ากับเวลาออก
PDO	ตัวเชื่อมระหว่าง PHP กับฐานข้อมูล	รองรับคำสั่งแบบเตรียมไว้ล่วงหน้า (prepared statement) ซึ่งเป็นวิธี ป้องกัน SQL Injection ที่ได้ผลจริง ดู บทที่ 6
JavaScript (ไม่ใช่เฟรม เวิร์ก)	ปุ่มเช็คอิน นาฬิกาจับ เวลา กราฟ ตัวกรอง และ หน้าต่างต่างๆ	งานฝั่งหน้าเว็บของระบบนี้ไม่ซับซ้อน พอที่จะคุ้มกับการติดตั้งเฟรม เวิร์ก และการไม่มีขั้นตอนแปลงไฟล์ ทำให้นำขึ้นเซิร์ฟเวอร์ได้ทันที
Tailwind CSS (แปลงไว้ แล้ว)	จัดรูปแบบหน้าเว็บทั้ง ระบบ	ใช้ไฟล์ที่แปลงเสร็จแล้วไฟล์เดียว ไม่ ต้องรันเครื่องมือแปลงบนเซิร์ฟเวอร์
IBM Plex Sans Thai	ฟอนต์ของหน้าเว็บและ ไฟล์รายงาน	เก็บไฟล์ฟอนต์ไว้ในเซิร์ฟเวอร์ตัวเอง หน้าเว็บจึงไม่ต้องต่ออินเทอร์เน็ต ออกไปข้างนอกเพื่อโหลดฟอนต์
mPDF	สร้างไฟล์ PDF ของ รายงานจากฝั่งเซิร์ฟเวอร์	รองรับภาษาไทยเมื่อใส่ฟอนต์ให้ และได้ไฟล์จริงที่กดบันทึกได้ ต่าง จากการสั่งพิมพ์ผ่านเบราว์เซอร์ที่ ผลลัพธ์ต่างกันไปตามเครื่อง

เครื่องมือ	ใช้ทำอะไร	เหตุผลที่เลือกสิ่งนี้
GD (มากับ PHP)	วาดกราฟแท่งและกราฟวงกลมเป็นรูปภาพเพื่อวางลงใน PDF	mPDF วาดกราฟที่สร้างจาก CSS สมัยใหม่ไม่ได้ จึงให้ GD วาดเป็นรูปสำเร็จแล้ว mPDF แค่วางรูปลงไป
PhpSpreadsheet	อ่านไฟล์รายชื่อนักศึกษาเข้าระบบครั้งแรก	ใช้เฉพาะสคริปต์นำเข้าข้อมูล ไม่ได้ทำงานตอนผู้ใช้เปิดเว็บ
Git และ GitHub	เก็บประวัติการแก้ไขโค้ดและเป็นต้นทางของการนำขึ้นเซิร์ฟเวอร์	ย้อนกลับไปเวอร์ชันก่อนหน้าได้เมื่อแก้แล้วพัง และเห็นได้ว่าแต่ละบรรทัดถูกแก้ไขเมื่อไหร่เพราะอะไร
GitHub Actions	อัปเดตไฟล์ขึ้นเซิร์ฟเวอร์อัตโนมัติตามลำดับที่กำหนด	ลดความผิดพลาดจากการลากไฟล์ขึ้นเองทีละไฟล์ และบังคับให้ซ่อมก่อนอัปเดตจริงทุกครั้ง
Hostinger	ผู้ให้บริการพื้นที่เว็บและโดเมน ntclibrary.com	ค่าบริการเหมาะกับโครงงานนักศึกษา และมีทั้ง PHP, MariaDB และใบรับรอง HTTPS ให้พร้อม
XAMPP	ชุดโปรแกรมสำหรับพัฒนาในเครื่องตนเอง	รวม PHP กับ MariaDB ไว้ในตัวเดียว ทำให้เครื่องที่ใช้พัฒนาเหมือนกับเซิร์ฟเวอร์จริงมากที่สุด

2.2 สิ่งที่ตั้งใจไม่ใช้ และเหตุผล

การเลือกไม่ใช้เครื่องมือบางอย่างเป็นการตัดสินใจเชิงออกแบบ เช่นเดียวกับการเลือกใช้ จึงขออธิบายไว้ด้วย

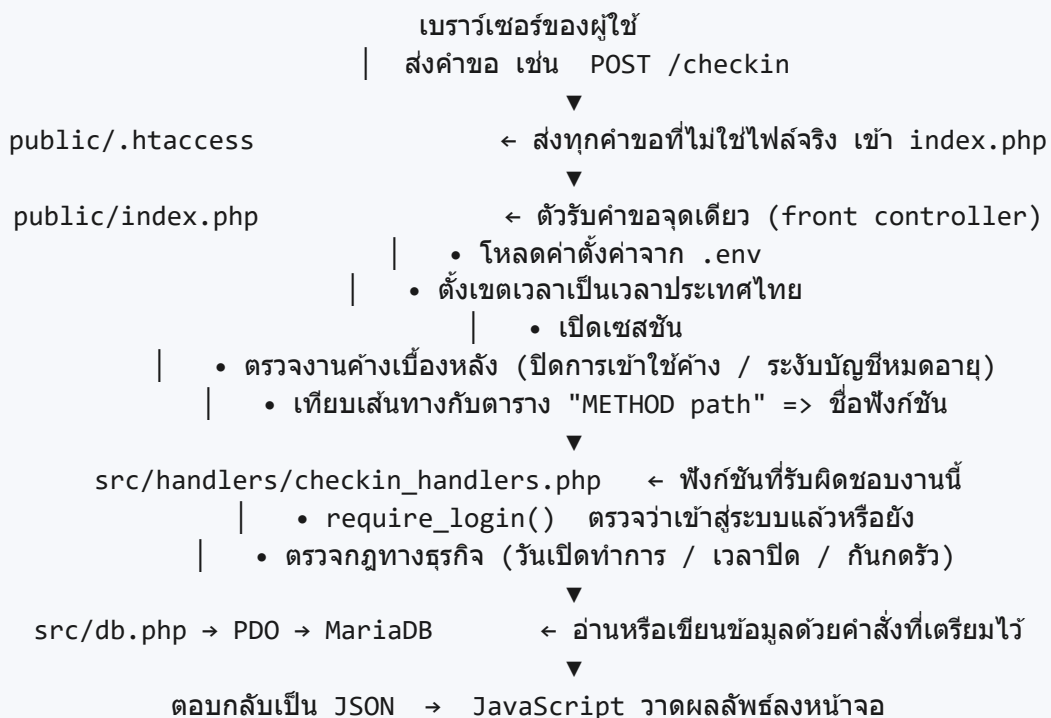
สิ่งที่ไม่ใช้	เหตุผล
เฟรมเวิร์ก PHP เช่น Laravel	ระบบมีเพียง 30 เส้นทาง ซึ่งเขียนเป็นตารางจับคู่ธรรมดาได้ครบ การใช้เฟรมเวิร์กจะเพิ่มไฟล์และขั้นตอนติดตั้งที่ต้องดูแลโดยไม่ได้แก้ปัญหาอะไรที่มีอยู่จริง

สิ่งที่ไม่ใช่	เหตุผล
เฟรมเวิร์กหน้าเว็บ เช่น React	เคยพัฒนาด้วย React มาก่อน แต่ต้องมีขั้นตอน แปลงไฟล์ก่อนนำขึ้นทุกครั้ง จึงย้ายมาเป็น PHP ที่ แสดงผลจากฝั่งเซิร์ฟเวอร์ เพื่อให้อัปเดตไฟล์แล้วใช้งาน ได้ทันที
Cron job สำหรับ งานตามเวลา	โฮสต์ที่ใช้จำกัดจำนวนงานตามเวลา ระบบจึงตรวจ งานค้าง เช่น การปิดการเข้าใช้ให้คนที่ล็อกด ในทุก ครั้งที่มีคนเปิดเว็บแทน ซึ่งเพียงพอกับปริมาณการใช้ งานระดับวิทยาลัยหนึ่งแห่ง
Docker	โฮสต์แบบแบ่งปันทรัพยากรไม่รองรับ และการ พัฒนาบน XAMPP ให้สภาพแวดล้อมที่ใกล้เคียงของ จริงอยู่แล้ว

บทที่ 3 สถาปัตยกรรมและการแบ่งงานของโค้ด

3.1 เส้นทางการทำงานของหนึ่งคำขอ

เมื่อผู้ใช้กดปุ่มบนหน้าเว็บหนึ่งครั้ง ระบบจะทำงานเรียงตามลำดับดังนี้



3.2 เหตุผลที่แยกโฟลเดอร์ public กับ src

โครงงานแบ่งไฟล์ออกเป็นสองกลุ่มอย่างชัดเจน กลุ่มแรกคือ `public/` ซึ่งเป็นโฟลเดอร์เดียวที่เว็บเซิร์ฟเวอร์เปิดให้เข้าถึงจากภายนอก และกลุ่มที่สองคือ `src/` ซึ่งอยู่นอกพื้นที่นั้น

เหตุผลคือความปลอดภัย ไฟล์ใน `src/` มีคำสั่งติดต่อฐานข้อมูล และตรรกะการตรวจสอบสิทธิ์ หากไฟล์เหล่านี้อยู่ในพื้นที่ที่เว็บเข้าถึงได้ และวันหนึ่งเซิร์ฟเวอร์ตั้งค่าผิดจนหยุดแปลไฟล์ PHP ผู้ที่พิมพ์ที่อยู่

ตรงเข้าไปจะเห็นซอร์สโค้ดทั้งไฟล์เป็นข้อความธรรมดา การวางไว้คนละชั้นทำให้ต่อให้เกิดเหตุนี้ก็ยังเข้าไม่ถึง

มีการตรวจสอบเรื่องนี้ทุกครั้งที่น่าขึ้นเซิร์ฟเวอร์

หลังอัปไฟล์เสร็จ ระบบนำขึ้นอัตโนมัติจะลองเรียก `/src/auth.php` จากภายนอก ถ้าได้รหัส 404 แปลว่าถูกต้อง แต่ถ้าได้ออย่างอื่นแปลว่าไฟล์ไปอยู่ผิดชั้นและระบบจะแจ้งเตือนทันที

3.3 หน้าที่ของแต่ละไฟล์

ไฟล์หรือโฟลเดอร์	รับผิดชอบอะไร
public/ — ส่วนที่เว็บเข้าถึงได้	
<code>index.php</code>	ตัวรับคำขอจุดเดียว เก็บตารางเส้นทางทั้ง 30 เส้น และเรียกฟังก์ชันที่รับผิดชอบ
<code>login.php</code> , <code>signup.php</code>	หน้าเข้าสู่ระบบและหน้าสมัครสมาชิก
<code>dashboard.php</code> , <code>profile.php</code>	หน้าหลักและหน้าข้อมูลส่วนตัวของนักศึกษา
<code>admin-dashboard.php</code> และ หน้า admin อื่น	หน้าภาพรวม ทำเนียบสมาชิก ผู้ที่กำลังใช้งาน และประวัติการเช็คชื่อของเจ้าหน้าที่
<code>partials/guard.php</code>	ยามเฝ้าหน้าเว็บ ตรวจสอบสิทธิ์ก่อนแสดงเนื้อหาใดๆ
<code>partials/</code> อื่นๆ	ชิ้นส่วนหน้าเว็บที่ใช้ซ้ำ เช่น แถบบน เมนูข้าง กล่องยืนยัน หน้าต่างประวัติ
<code>assets/</code>	ไฟล์ CSS, JavaScript, ฟอนต์ และรูปภาพ
src/ — ส่วนที่เว็บเข้าถึงโดยตรงไม่ได้	

ไฟล์หรือโฟลเดอร์	รับผิดชอบอะไร
<code>env.php</code>	อ่านค่าตั้งค่าจากไฟล์ <code>.env</code>
<code>db.php</code>	เปิดการเชื่อมต่อฐานข้อมูล พร้อมตั้งเขตเวลาและชุดอักขระให้ถูกต้อง
<code>auth.php</code>	ตั้งค่าเซสชัน และฟังก์ชันตรวจสอบสิทธิ์ <code>require_login()</code> กับ <code>require_admin()</code>
<code>constants.php</code>	ค่าคงที่และกฎการตรวจสอบความถูกต้องของข้อมูลที่ใช้กรอก
<code>helpers.php</code>	ฟังก์ชันช่วยทั่วไป เช่น การตอบกลับเป็น JSON เวลาปิดห้องสมุด และวันเปิดทำการ
<code>rate_limit.php</code>	จำกัดจำนวนครั้งที่กรอกรหัสผ่านผิด
<code>handlers/</code>	ตรรกะของแต่ละกลุ่มงาน แยกเป็น <code>auth</code> , <code>checkin</code> , <code>profile</code> , <code>admin</code> และ <code>announcement</code>
<code>reports/</code>	ระบบรายงานทั้ง 8 แบบ พร้อมตัวช่วยที่ใช้ร่วมกัน
ไฟล์อื่นที่อยู่นอกทั้งสองโฟลเดอร์	
<code>.env</code>	รหัสผ่านฐานข้อมูลและค่าตั้งค่าเฉพาะเครื่อง ไม่ถูกเก็บลง Git
<code>schema.sql</code>	คำสั่งสร้างโครงสร้างฐานข้อมูลทั้งหมด
<code>tests/smoke_test.php</code>	ชุดทดสอบระบบตั้งแต่ต้นจนจบ 27 กรณี
<code>fonts/</code>	ไฟล์ฟอนต์สำหรับสร้าง PDF ซึ่งคนละไฟล์กับฟอนต์ของหน้าเว็บ

3.4 เหตุผลที่ใช้ตัวรับค่าของจุดเดียว

ทุกคำขอที่เข้ามหาระบบจะผ่าน `public/index.php` เพียงไฟล์เดียว ไม่มีการเรียกไฟล์ PHP อื่นโดยตรง ข้อดีคือสิ่งที่ต้องทำก่อนเสมอถูกเขียนไว้ที่เดียว ได้แก่การโหลดค่าตั้งค่า การตั้งเขตเวลา การเปิดเซสชัน และการตรวจงานค้างเบื้องหลัง หากแยกเป็นหลายไฟล์

ทางเข้า จะต้องเขียนขั้นตอนเหล่านี้ซ้ำทุกไฟล์ และวันที่ลืมเขียนในไฟล์ใดไฟล์หนึ่งคือวันที่เกิดข้อผิดพลาดที่หาสาเหตุยาก

ตารางเส้นทางเขียนในรูปแบบ "วิธีการ ที่อยู่" => ชื่อฟังก์ชัน เช่น
'POST /checkin' => 'handle_checkin' ถ้าคำขอไม่ตรงกับรายการใดเลย ระบบจะตอบรหัส 404 ทันทีโดยไม่ทำงานอื่นต่อ

บทที่ 4 ฐานข้อมูล

4.1 ตารางทั้งหมด 6 ตาราง

4.1.1 ตาราง students — ข้อมูลนักศึกษา (11 คอลัมน์)

เก็บข้อมูลตัวบุคคล ไม่เกี่ยวกับการเข้าสู่ระบบ

คอลัมน์	ชนิดข้อมูล	ความหมาย
student_id	VARCHAR(20) คีย์หลัก	รหัสนักศึกษา ใช้เป็นตัวระบุตัวตนหลัก
prefix	VARCHAR(20)	คำนำหน้าชื่อ รับเฉพาะ นาย นาง นางสาว
gender	ENUM(male, female) ว่างได้	เพศ ว่างได้เพราะข้อมูลที่นำเข้าจากรายชื่อไม่มีค่านี้
first_name	VARCHAR(100)	ชื่อ
last_name	VARCHAR(100)	นามสกุล
department	VARCHAR(100)	แผนกวิชา รับได้เฉพาะ 14 แผนกที่กำหนดไว้
level	VARCHAR(10)	ระดับชั้น ปวช. หรือ ปวส.
year_level	VARCHAR(20)	ชั้นปี ปวช. มี 1–3 ส่วน ปวส. มี 1–2
room	VARCHAR(10)	ห้องเรียน
semester	VARCHAR(10)	ภาคเรียน
academic_year	VARCHAR(10)	ปีการศึกษา

4.1.2 ตาราง users — บัญชีเข้าสู่ระบบ (8 คอลัมน์)

เก็บสิ่งที่เกี่ยวกับการเข้าสู่ระบบและสิทธิ์ ทั้งบัญชีนักศึกษาและบัญชีเจ้าหน้าที่อยู่ในตารางนี้ ดูเหตุผลในหัวข้อ 4.3

คอลัมน์	ชนิดข้อมูล	ความหมาย
<code>user_id</code>	INT คีย์หลัก เพิ่มเอง	เลขบัญชีที่ระบบสร้างให้
<code>username</code>	VARCHAR(50) ห้ามซ้ำ	ชื่อผู้ใช้ นักศึกษาใช้รหัสนักศึกษาเป็นชื่อผู้ใช้
<code>password_hash</code>	VARCHAR(255)	ค่าที่ผ่านการแปลงด้วย bcrypt แล้ว ไม่ใช้รหัสผ่านจริง ดูปทที่ 6
<code>role</code>	ENUM(student, admin)	คอลัมน์ที่ใช้แยกว่าใครเป็นนักศึกษา ใครเป็นเจ้าหน้าที่
<code>student_id</code>	VARCHAR(20) คีย์นอก	ชี้ไปที่ตาราง students บัญชีเจ้าหน้าที่มีค่าเป็นว่าง
<code>id_card_photo_path</code>	VARCHAR(255)	ที่อยู่ไฟล์รูปบัตรนักศึกษา
<code>account_status</code>	ENUM(pending, approved, retired)	สถานะบัญชี สมัครใหม่จะได้ <code>approved</code> ทันที ส่วน <code>retired</code> ระบบตั้งให้เองเมื่อครบ 1 ปีการศึกษา
<code>created_at</code>	DATETIME	วันเวลาที่สร้างบัญชี ใช้นับอายุบัญชี

4.1.3 ตาราง `checkin_logs` — ประวัติการเข้า-ออก (6 คอลัมน์)

คอลัมน์	ชนิดข้อมูล	ความหมาย
<code>log_id</code>	INT คีย์หลัก เพิ่มเอง	เลขลำดับของเหตุการณ์
<code>user_id</code>	INT คีย์นอก	ชี้ไปที่ตาราง users
<code>timestamp</code>	DATETIME	วันเวลาที่เกิดเหตุการณ์
<code>type</code>	ENUM(in, out)	เข้า หรือ ออก
<code>planned_check_out_at</code>	DATETIME ว่างได้	เวลาที่ตั้งใจจะออก บันทึกเฉพาะตอนเช็คอิน ค่าว่างแปลว่าอยู่จนกว่าจะปิด

คอลัมน์	ชนิดข้อมูล	ความหมาย
checkout_source	ENUM(manual, auto, admin_forced)	ใครเป็นคนบันทึกออก ผู้ใช้กดเอง ระบบปิดให้ หรือเจ้าหน้าที่บังคับปิด

4.1.4 ตาราง login_attempts — ประวัติการรอกหักสผิด (4 คอลัมน์)

คอลัมน์	ชนิดข้อมูล	ความหมาย
id	INT คีย์หลัก เพิ่มเอง	เลขลำดับ
ip	VARCHAR(45)	หมายเลขเครื่องที่ส่งคำขอเข้ามา
username	VARCHAR(50)	ชื่อผู้ใช้ที่ถูกเดา
attempted_at	DATETIME	เวลาที่กรอกผิด

บันทึกเฉพาะครั้งที่กรอกผิดเท่านั้น การเข้าสู่ระบบสำเร็จไม่ถูกนับ ดูเหตุผลในหัวข้อ 6.7

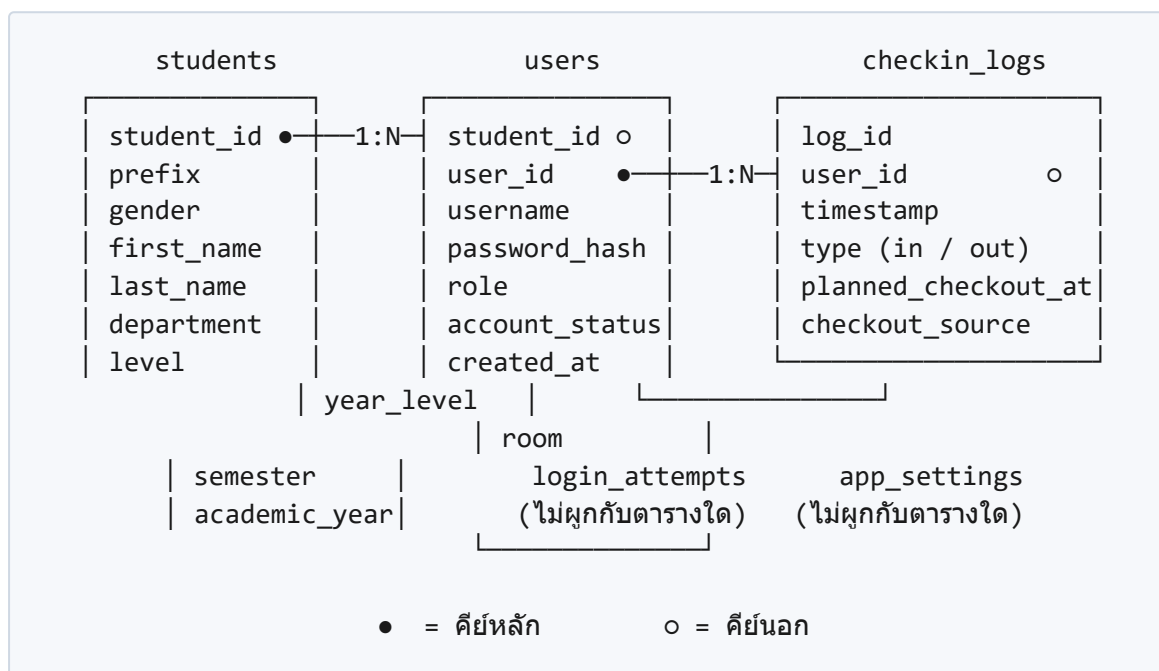
4.1.5 ตาราง app_settings — ค่าตั้งค่าของระบบ (4 คอลัมน์)

คอลัมน์	ชนิดข้อมูล	ความหมาย
setting_key	VARCHAR(50) คีย์หลัก	ชื่อค่า ปัจจุบันใช้อยู่ 2 ค่าคือข้อความประกาศและสถานะเปิด-ปิดการแสดงผลประกาศ
setting_value	TEXT	ค่าที่เก็บ
updated_at	DATETIME	เวลาที่แก้ล่าสุด
updated_by	VARCHAR(50)	ชื่อผู้ใช้ของเจ้าหน้าที่ที่แก้ล่าสุด เก็บเป็นข้อความไม่ใช่คีย์นอก เพื่อให้ประกาศยังอยู่แม้บัญชีนั้นถูกลบ

4.1.6 ตาราง schema_migrations — บันทึกการปรับโครงสร้าง (2 คอลัมน์)

เก็บชื่อไฟล์ปรับโครงสร้างฐานข้อมูลที่รันไปแล้ว เพื่อกันไม่ให้ถูกรันซ้ำ

4.2 ความสัมพันธ์ระหว่างตาราง



ความสัมพันธ์หลักมีสองเส้น เส้นแรกคือนักศึกษาหนึ่งคนมีบัญชีได้ตั้งแต่หนึ่งบัญชีขึ้นไป และเส้นที่สองคือบัญชีหนึ่งบัญชีมีประวัติเข้า-ออกได้หลายรายการ ในทางเทคนิคเส้นแรกเป็นแบบหนึ่งต่อกลุ่ม เพราะคอลัมน์ `users.student_id` ไม่ได้กำหนดว่าห้ามซ้ำ แม้ในทางปฏิบัติจะใช้แบบหนึ่งต่อหนึ่งก็ตาม

4.3 แอดมินกับผู้ใช้อยู่ตารางเดียวกันหรือแยกกัน

คำตอบ: อยู่ตารางเดียวกัน คือตาราง `users` แล้วใช้คอลัมน์ `role` เป็นตัวแยก

ไม่ได้แยกเป็นตาราง `admins` กับ `students` คนละตาราง แต่ใช้ตาราง `users` ตารางเดียว โดยมีคอลัมน์ `role` ชนิด ENUM ที่รับได้เพียงสองค่าคือ `'student'` กับ `'admin'`

เหตุผลที่เลือกแบบนี้

1. สิ่งที่ทั้งสองกลุ่มมีเหมือนกันคือ “การเป็นบัญชีที่เข้าสู่ระบบได้” ทั้งเจ้าหน้าที่และนักศึกษาต่างต้องมีชื่อผู้ใช้ รหัสผ่านที่ผ่านการเข้ารหัส สถานะบัญชี และวันที่สร้าง ถ้าแยกเป็นสองตารางจะต้องมีคอลัมน์ชุดเดียวกันนี้ซ้ำกันทั้งสองที่ ซึ่งขัดกับหลักการทำให้เป็นบรรทัดฐาน (Normalization)
2. การเข้าสู่ระบบจะกลายเป็นเรื่องยุ่งยากทันทีถ้าแยกตาราง ระบบจะต้องค้นหาชื่อผู้ใช้ในตารางหนึ่งก่อน ถ้าไม่เจอค่อยไปค้นอีกตาราง หรือไม่ก็ต้องถามผู้ใช้นี้ก่อนว่าเป็นบัญชีประเภทไหน ซึ่งเป็นคำถามที่ระบบควรตอบเองได้ เมื่ออยู่ตารางเดียวกัน คำสั่งค้นหาเพียงคำสั่งเดียวก็ได้ทั้งบัญชีและสิทธิมาพร้อมกัน
3. ประวัติการเข้าใช้ชี้ไปที่ตารางเดียว ตาราง `checkin_logs` มีคีย์นอกชี้ไปที่ `users.user_id` ถ้าแยกบัญชีเป็นสองตาราง คีย์นอกนี้จะชี้ไปที่ใดที่หนึ่งไม่ได้ ต้องเพิ่มคอลัมน์บอกประเภทกำกับ ซึ่งเป็นรูปแบบที่ฐานข้อมูลบังคับความถูกต้องให้ไม่ได้
4. การเปลี่ยนสิทธิ์ทำได้ด้วยการแก้ค่าเดียว เมื่อนักศึกษาช่วยงานห้องสมุดและต้องได้สิทธิ์เจ้าหน้าที่ ระบบเพียงเปลี่ยนค่า `role` จาก `student` เป็น `admin` ถ้าแยกตารางจะต้องย้ายข้อมูลข้ามตาราง ซึ่งเสี่ยงที่ประวัติเดิมจะขาดการเชื่อมโยง

ข้อมูลส่วนตัวจึงถูกแยกออกไปอีกตารางแทน

สิ่งที่แยกตารางจริงคือ **ข้อมูลส่วนบุคคล** ซึ่งอยู่ในตาราง `students` เหตุผลคือข้อมูลชุดนี้มีเฉพาะนักศึกษาเท่านั้น เจ้าหน้าที่ไม่มีแผนกวิชา ไม่มีชั้นปี ไม่มีห้องเรียน หากยัดรวมไว้ในตาราง `users` ทุกแถวของเจ้าหน้าที่จะมีคอลัมน์ที่ว่างเปล่าติดอยู่หลายคอลัมน์ตลอดไป

สรุปหลักการที่ใช้แบ่งคือ ตาราง `users` ตอบคำถามว่า “ใคร เข้าระบบได้และทำอะไรได้บ้าง” ส่วนตาราง `students` ตอบคำถามว่า “คนคนนี้เป็นใคร” ซึ่งเป็นคำถามคนละเรื่องกัน

4.4 เหตุผลที่เก็บการเช็คชื่อเป็น “เหตุการณ์”

ตาราง `checkin_logs` เก็บหนึ่งแถวต่อหนึ่งเหตุการณ์ คือเข้าหนึ่งแถว ออกอีกหนึ่งแถว ไม่ได้เก็บหนึ่งแถวต่อหนึ่งการมา ที่มีทั้งคอลัมน์เวลาเข้าและเวลาออกอยู่ในแถวเดียวกัน

เหตุผลคือขณะที่นักศึกษาเช็คอิน ระบบยังไม่รู้ว่าเขาจะออกเมื่อไร ถ้าเก็บแบบหนึ่งแถวต่อหนึ่งการมา จะต้องสร้างแถวที่มีคอลัมน์เวลาออกกว้างไว้ก่อน แล้วค่อยกลับมาแก้ทีหลัง ซึ่งหมายความว่าทุกคำสั่งที่อ่านตารางนี้ต้องเผื่อกรณี “แถวที่ยังไม่สมบูรณ์” เสมอ การเก็บเป็นเหตุการณ์ทำให้ทุกแถวสมบูรณ์ตั้งแต่วันที่เขียนลงไป และไม่มี การแก้ไขแถวเดิมเลย มีแต่การเพิ่มแถวใหม่

เมื่อต้องแสดงผลเป็น “การมา 1 ครั้ง” เช่นในหน้าประวัติของนักศึกษา ระบบจะจับคู่แถวเข้ากับแถวออกด้วยฟังก์ชันหน้าต่างของ SQL ชื่อ `LEAD()` ซึ่งทำให้แต่ละแถวมองเห็นแถวถัดไปของผู้ใช้คนเดียวกันได้

4.5 ดัชนีที่สร้างไว้และเหตุผล

ดัชนี	แก้ปัญหอะไร
<code>idx_checkin_logs_timestamp</code>	รายงานทุกแบบกรองด้วยช่วงวันที่ ถ้าไม่มีดัชนีนี้ ฐานข้อมูลต้องไล่อ่านทั้งตารางทุกครั้งที่เปิดรายงาน
<code>idx_checkin_logs_user_time</code>	ใช้กับการค้นประวัติของผู้ใช้รายคนเรียงจากใหม่ไปเก่า ซึ่งหน้าหลักและระบบปิดการเข้าใช้อัตโนมัติเรียกใช้บ่อยที่สุด
<code>idx_login_attempts_ip_time</code> และ <code>idx_login_attempts_ip_user_time</code>	ใช้นับจำนวนครั้งที่กรอกรหัสผิดภายในหนึ่งนาที ซึ่งถูกเรียกทุกครั้งที่มีคนกดเข้าสู่ระบบ

คีย์นอกของ MariaDB สร้างดัชนีให้เฉพาะคอลัมน์ที่เป็นคีย์นอกเท่านั้น จึงต้องเพิ่มดัชนีผสมข้างต้นเอง

บทที่ 5 ระบบสมาชิกและการแบ่งสิทธิ์

5.1 คอลัมน์ที่ใช้แยกสิทธิ์

ระบบใช้สองคอลัมน์ในตาราง `users` ตอบคำถามคนละข้อ

คอลัมน์	ค่าที่เป็นไปได้	ตอบคำถามว่า
<code>role</code>	<code>student</code> / <code>admin</code>	บัญชีนี้ทำอะไรได้บ้าง
<code>account_status</code>	<code>pending</code> / <code>approved</code> / <code>retired</code>	บัญชีนี้ยังเข้าระบบได้อยู่หรือไม่

บัญชีที่สมัครใหม่จะได้สถานะ `approved` ทันที ใช้งานได้เลยโดยไม่ต้องรอเจ้าหน้าที่กดอนุมัติ ส่วนค่า `pending` มีไว้ให้เจ้าหน้าที่ตั้งเองเมื่อต้องการระงับบัญชีชั่วคราว และค่า `retired` ระบบตั้งให้เองเมื่อบัญชีมีอายุครบ 1 ปีการศึกษานับจากวันที่สร้าง

5.2 ตัวกรองสิทธิ์อยู่ตรงไหนในโค้ด

มีการตรวจสอบสิทธิ์อยู่ 3 จุด และทั้งสามจุดทำงานที่ฝั่งเซิร์ฟเวอร์ทั้งหมด

จุดที่ 1 — ตอนเข้าสู่ระบบ บันทึกสิทธิ์ลงเซสชัน

```
// src/handlers/auth_handlers.php
$_SESSION['user_id'] = $user['user_id'];
$_SESSION['role']    = $user['role'];      // ← เก็บสิทธิ์ไว้ที่ฝั่งเซิร์ฟเวอร์
```

ค่า `role` ถูกอ่านจากฐานข้อมูลแล้วเก็บไว้ในเซสชันซึ่งอยู่ที่เซิร์ฟเวอร์ ผู้ใช้แก้ค่านี้จากเบราว์เซอร์ไม่ได้

จุดที่ 2 — ตอนเรียก API ทุกเส้นทาง

```
// src/auth.php
function require_login(): void {
    if (!isset($_SESSION['user_id'])) {
        json_error('กรุณาเข้าสู่ระบบ', 401);
    }
}

function require_admin(): void {
    if (($SESSION['role'] ?? null) !== 'admin') {
        json_error('ต้องใช้สิทธิ์แอดมิน', 403);
    }
}
```

ทุกฟังก์ชันที่รับผิดชอบงานของเจ้าหน้าที่จะเรียกสองบรรทัดนี้เป็นสิ่งแรกเสมอ เช่น

```
function handle_admin_members(): void {
    require_login();
    require_admin();           // ← นักศึกษาที่ยังมาตรงนี้จะได้ 403
    ...
}
```

จุดที่ 3 — ตอนเปิดหน้าเว็บ

```
// public/partials/guard.php
if (!isset($_SESSION['user_id'])) {
    header('Location: /login'); // ยังไม่เข้าสู่ระบบ → ส่งไปหน้า
    ล็อกอิน
    exit;
}
if (!empty($requireAdmin) && ($_SESSION['role'] ?? null) !==
    'admin') {
    header('Location: /dashboard'); // เป็นนักศึกษาแต่จะเข้าหน้าแอด
    มิน → ส่งกลับ
    exit;
}
```

ไฟล์นี้ถูกเรียกเป็นบรรทัดแรกสุดของทุกหน้าที่ต้องเข้าสู่ระบบ โดยหน้าของเจ้าหน้าที่จะตั้งค่า `$requireAdmin = true;` ไว้ก่อนเรียก

จุดสำคัญ: การตรวจสอบสิทธิ์ไม่ได้อยู่ที่ฝั่งเบราว์เซอร์

ในหน้าเว็บมีการซ่อนเมนูที่ผู้ใช้ไม่มีสิทธิ์ใช้อยู่ด้วย แต่นั่นเป็นเพียงความสะดวกไม่ใช่ว่าการป้องกัน เพราะสิ่งที่ซ่อนด้วย JavaScript ผู้ใช้เปิดเครื่องมือของเบราว์เซอร์แล้วเอากลับมาได้เสมอ ตัวที่ป้องกันจริงคือสามจุดข้างต้นซึ่งอยู่ที่เซิร์ฟเวอร์ ต่อให้ผู้ใช้พิมพ์ที่อยู่หน้าแอดมินตรงๆ หรือยิงคำขอไปที่ API โดยไม่ผ่านหน้าเว็บ ก็ยังถูกปฏิเสธอยู่ดี

5.3 เหตุผลที่แยกหน้าของแอดมินกับของผู้ใช้

ระบบไม่ได้ใช้หน้าเดียวกันแล้วซ่อนบางปุ่มตามสิทธิ์ แต่แยกเป็นคนละหน้าไปเลย ด้วยเหตุผล 4 ข้อ

1. **งานที่ทำคนละเรื่องกันโดยสิ้นเชิง** นักศึกษาเปิดเว็บมาเพื่อกดปุ่มเดียวคือเช็คอิน หน้าจอจึงควรมีปุ่มนั้นเด่นที่สุดและไม่มีอะไรมารบกวน

บกววน ส่วนเจ้าหน้าที่เปิดมาเพื่อดูตัวเลขสรุป ค้นหาสมาชิก และ
ออกรายงาน ซึ่งต้องการตาราง ตัวกรอง และกราฟ ถ้ารวมสอง
อย่างไว้ในหน้าเดียว ทั้งสองกลุ่มจะได้หน้าจอที่รกเกินความ
จำเป็นของตน

2. **ลดโอกาสพลาดด้านความปลอดภัย** หน้าที่ยรวมทุกอย่างไว้แล้ว
ซ่อนตามสิทธิ์ มีความเสี่ยงที่วันหนึ่งจะลืมใส่เงื่อนไขซ่อนกับปุ่ม
ใหม่ที่เพิ่งเพิ่ม เมื่อแยกไฟล์แล้ว หน้าแอดมินทุกไฟล์ตั้ง
`$requireAdmin = true;` ไว้ที่บรรทัดบนสุด สิ่งที่อยู่ในไฟล์นั้นจึง
ถูกป้องกันทั้งไฟล์โดยอัตโนมัติ
3. **โหลดเร็วกว่า** หน้าของนักศึกษาไม่ต้องโหลดโค้ดกราฟ ตัวกรอง
หรือระบบตาราง ซึ่งสำคัญมากเพราะนักศึกษาส่วนใหญ่เปิดผ่าน
โทรศัพท์บนสัญญาณของวิทยาลัย
4. **แก้ไขต่อได้ง่ายกว่า** การแก้หน้าเจ้าหน้าที่ไม่มีทางทำให้ปุ่มเช็ค
อินของนักศึกษาพัง เพราะอยู่คนละไฟล์กัน

อย่างไรก็ตาม สิ่งที่ใช้ร่วมกันได้ก็ยังใช้ร่วมกัน เช่น แถบบนสุด
กล่องยืนยัน และไฟล์ CSS ถูกเก็บไว้ในโฟลเดอร์ `partials/` และ
`assets/` ที่ทั้งสองฝั่งเรียกใช้ จึงไม่ได้เขียนซ้ำสองรอบ

5.4 การกันไม่ให้ระบบเหลือแอดมินศูนย์คน

มีสามการกระทำที่อาจทำให้ระบบไม่เหลือผู้ดูแลเลย ได้แก่การ
ลดสิทธิ์เจ้าหน้าที่ลงเป็นนักศึกษา การระงับบัญชีเจ้าหน้าที่ และการ
ลบบัญชีเจ้าหน้าที่ ทั้งสามอย่างนี้ระบบจะนับก่อนเสมอว่ายังมีเจ้า
หน้าที่คนอื่นที่ใช้งานได้เหลืออยู่หรือไม่ ถ้าไม่เหลือจะปฏิเสธการกระทำ
นั้น เพราะหากปล่อยให้เกิดขึ้น จะไม่มีใครเข้าไปแก้กลับได้อีกเลย
นอกจากแก้ตรงในฐานข้อมูล

บทที่ 6 ความปลอดภัย

6.1 การเก็บรหัสผ่าน

คำตอบสั้น: ระบบไม่เคยเก็บรหัสผ่านจริง เก็บเฉพาะค่าที่แปลงด้วย bcrypt แล้ว

ไม่มีใครดูรหัสผ่านของผู้ใช้ได้ ไม่ว่าจะเป็นเจ้าหน้าที่ ผู้ดูแลระบบ หรือผู้พัฒนาเอง เพราะสิ่งที่อยู่ในฐานข้อมูลไม่ใช่รหัสผ่าน และแปลงกลับเป็นรหัสผ่านไม่ได้

สิ่งที่เกิดขึ้นตอนสมัครสมาชิก

```
// src/handlers/auth_handlers.php
$passwordHash = password_hash($password, PASSWORD_BCRYPT);
// เก็บ $passwordHash ลงฐานข้อมูล ส่วน $password ตัวจริงถูกทิ้งไปพร้อมกับค่าของมัน
```

สิ่งที่เห็นในฐานข้อมูลจริง

ถ้าเปิดตาราง `users` ดู คอลัมน์ `password_hash` จะเห็นเป็นแบบนี้

```
$2y$10$R5mw3n3SWxa9H2jvufa7n.A6rY6y2CkRS1L5k1JFc1auNRjK.oJK
```

ข้อความยาว 60 ตัวอักษรนี้อ่านได้ดังนี้

ส่วน	ค่าในตัวอย่าง	ความหมาย
อัลกอริทึม	<code>\$2y\$</code>	บอกว่าใช้ bcrypt
ความหนัก	<code>10</code>	คำนวณซ้ำ $2^{10} = 1,024$ รอบ ยิ่งสูงยิ่งเดายาก แต่ก็ยิ่งช้า
เกลือ (salt)	22 ตัวอักษร ถัดมา	ค่าสุ่มที่ระบบสร้างให้ใหม่ทุกครั้ง

ส่วน	ค่าในตัวอย่าง	ความหมาย
ค่าที่แปลงแล้ว	31 ตัวอักษร สุดท้าย	ผลลัพธ์ของการแปลง

แล้วตรวจรหัสผ่านตอนเข้าสู่ระบบได้อย่างไร

```
if (!password_verify($password, $user['password_hash'])) {
    // รหัสผ่านไม่ถูกต้อง
}
```

ฟังก์ชัน `password_verify()` จะอ่านเกลือออกมาจากค่าที่เก็บไว้ แล้วนำรหัสผ่านที่ผู้ใช้เพิ่งพิมพ์มาแปลงด้วยเกลือชุดเดียวกัน ถ้าผลลัพธ์ตรงกันแปลว่ารหัสถูก กระบวนการนี้เป็นการแปลงไปข้างหน้าแล้วเทียบ ไม่ใช่การถอดกลับ

ทำไมต้องมีเกลือ

เกลือคือค่าสุ่มที่ต่างกันทุกบัญชี ผลคือผู้ใช้สองคนที่ตั้งรหัสผ่านเหมือนกันเป๊ะ จะได้ค่าในฐานข้อมูลที่ไม่เหมือนกันเลย ทำให้ผู้ที่ขโมยฐานข้อมูลไปไม่สามารถเดาได้ว่าใครใช้รหัสผ่านซ้ำกับใคร และไม่สามารถใช้ตารางค่าที่คำนวณไว้ล่วงหน้า (rainbow table) มาเทียบทีเดียวทั้งฐานข้อมูลได้

6.2 เหตุผลที่ไม่ใช้ MD5 หรือ SHA-1

วิธี	ความเร็ว	ทำไมจึงไม่เหมาะกับรหัสผ่าน
MD5	เร็วมาก	การ์ดจอสมัยใหม่คำนวณได้หลายพันล้านค่าต่อวินาที รหัสผ่านทั่วไปจึงถูกไล่เดาจนครบในเวลาไม่นาน อีกทั้งยังพบวิธีสร้างข้อมูลสองชุดที่ให้ค่าเดียวกันแล้ว

วิธี	ความเร็ว	ทำไมจึงไม่เหมาะกับรหัสผ่าน
SHA-1	เร็วมาก	ปัญหาเดียวกับ MD5 และถูกประกาศเลิกใช้ในงานด้านความปลอดภัยแล้ว
bcrypt	ช้าโดยตั้งใจ	ออกแบบมาให้คำนวณช้าและปรับให้ช้าลงอีกได้เมื่อคอมพิวเตอร์เร็วขึ้น พร้อมใส่เกลือให้อัตโนมัติ ผู้เดารหัสจึงเดาได้ช้ามากจนไม่คุ้ม

หลักการที่ต่างกันคือ MD5 และ SHA-1 ถูกออกแบบมาเพื่อ **ตรวจสอบความถูกต้องของไฟล์** ซึ่งงานนั้นยิ่งเร็วยิ่งดี แต่การเก็บรหัสผ่านต้องการสิ่งตรงกันข้าม คือยิ่งช้ายิ่งปลอดภัย

6.3 การจัดการเมื่อผู้ใช้ลืมรหัสผ่าน

เนื่องจากระบบไม่ได้เก็บรหัสผ่านจริง จึงไม่มีทาง “ดูรหัสเดิมให้” ได้ วิธีที่ระบบใช้คือเจ้าหน้าที่กดปุ่มรีเซ็ตรหัสผ่านในทำเนียบสมาชิก ระบบจะสุ่มรหัสชั่วคราวขึ้นมาใหม่ แปลงด้วย bcrypt เก็บทับค่าเดิม แล้วแสดงรหัสชั่วคราวนั้นบนหน้าจอครั้งเดียวเพื่อให้เจ้าหน้าที่บอกนักศึกษา

รายละเอียดเล็กน้อยที่ตั้งใจออกแบบไว้คือ รหัสที่สุ่มจะตัดตัว **อักษรที่สับสนออก** เช่น เลขศูนย์กับตัวโอ และเลขหนึ่งกับตัวแอลเล็ก และตัวไออักษรใหญ่ เพราะรหัสนี้ต้องถูกอ่านออกเสียงให้ฟังกัน

6.4 การป้องกัน SQL Injection

SQL Injection คือการที่ผู้ใช้พิมพ์คำสั่ง SQL ปนเข้ามาในช่องกรอกข้อมูล แล้วคำสั่งนั้นถูกนำไปทำงานจริง ระบบนี้ป้องกันด้วยการใช้คำสั่งแบบเตรียมไว้ล่วงหน้าของ PDO ทุกจุดที่มีการติดต่อฐานข้อมูล

```
// วิธีที่ระบบใช้ – ปลอดภัย
$stmt = $conn->prepare('SELECT * FROM users WHERE username =
?');
$stmt->execute([$username]);

// วิธีที่ไม่ปลอดภัย และไม่มีอยู่ในโค้ดของระบบนี้
$conn->query("SELECT * FROM users WHERE username =
'$username'");
```

ความต่างคือ วิธีแรกส่ง**โครงสร้าง**ไปให้ฐานข้อมูลแปลงก่อน แล้วจึงส่งค่าตามไปที่หลังแยกกันคนละรอบ ฐานข้อมูลจึงถือว่าสิ่งที่ผู้ใช้พิมพ์เป็นข้อมูลเสมอ ไม่ว่าจะพิมพ์อะไรลงไปก็ตาม ส่วนวิธีที่สองเป็นการต่อข้อความเข้าไปในคำสั่งก่อนส่ง ฐานข้อมูลจึงแยกไม่ออกว่าส่วนใดคือคำสั่งและส่วนใดคือข้อมูล

นอกจากนี้ระบบยังปิดโหมดจำลองคำสั่ง (`ATTR_EMULATE_PREPARES = false`) เพื่อให้เป็นการเตรียมคำสั่งที่ฝั่งฐานข้อมูลจริง ไม่ใช่ให้ PHP ต่อข้อความให้แล้วส่งไปทั้งก้อน

6.5 การป้องกัน XSS

XSS คือการที่ผู้ใช้ฝากโค้ด JavaScript ไว้ในข้อมูล เช่น ตั้งชื่อตัวเองเป็นแท็กสคริปต์ แล้วโค้ดนั้นถูกนำไปแสดงบนหน้าจอของคนอื่นจนทำงานจริง ระบบป้องกันสองชั้น

1. **ชั้นแสดงผล** ข้อมูลทุกอย่างที่มาจากผู้ใช้จะถูกแปลงอักขระพิเศษก่อนแสดงเสมอ ฝั่ง PHP ใช้ `htmlspecialchars()` และฝั่ง JavaScript ใช้ฟังก์ชัน `escapeHtml()` หรือกำหนดค่าผ่าน `textContent` ซึ่งเบราว์เซอร์ถือว่าเป็นข้อความล้วนไม่ใช่โค้ด
2. **ชั้นรับข้อมูล** ช่องชื่อ นามสกุล และข้อความประกาศ ปฏิเสธเครื่องหมาย `<` และ `>` ตั้งแต่ตอนบันทึก ทำให้ข้อมูลอันตรายไม่มีโอกาสเข้าไปอยู่ในฐานข้อมูลตั้งแต่แรก

6.6 การจัดการเซสชัน

มาตรการ	ป้องกันอะไร
<code>httponly = true</code>	JavaScript อ่านคุกกี้เซสชันไม่ได้ ต่อให้มีช่องโหว่ XSS หลุดรอดไป ก็ขโมยคุกกี้ออกไปไม่ได้
<code>secure = true</code> (เฉพาะเครื่องจริง)	คุกกี้ถูกส่งเฉพาะบน HTTPS จึงดักอ่านระหว่างทางไม่ได้
<code>samesite = Lax</code>	เบราว์เซอร์ไม่แนบคุกกี้ไปกับคำขอที่เว็บอื่นเป็นคนส่ง ซึ่งเป็นการป้องกัน CSRF ระดับพื้นฐาน
<code>session_regenerate_id(true)</code>	ออกรหัสเซสชันใหม่ทันทีที่เข้าสู่ระบบสำเร็จ ป้องกันการโจมตีแบบวางรหัสเซสชันไว้ล่วงหน้า (session fixation)
อายุ 30 วัน	เดิมตั้งไว้ให้หมดอายุเมื่อปิดเบราว์เซอร์ ซึ่งบนโทรศัพท์พบว่าถูกไล่ออกจากระบบบ่อยมาก จึงยืดเป็น 30 วัน โดยตั้งอายุฝั่งเซิร์ฟเวอร์ให้เท่ากันด้วย

6.7 การป้องกันการเดารหัสผ่าน

ทุกครั้งที่กรอกรหัสผ่านผิด ระบบจะบันทึกลงตาราง `login_attempts` แล้วนับย้อนหลัง 1 นาที ถ้าเกินเกณฑ์จะปฏิเสธคำขอด้วยรหัส 429

เกณฑ์	จำนวน	เหตุผล
ผิดต่อ (เครื่อง + ชื่อผู้ใช้) หนึ่งคู่	5 ครั้ง/ นาที	หยุดการเดารหัสของบัญชีใด บัญชีหนึ่งโดยเฉพาะ
ผิดต่อเครื่องเดียว รวมทุกชื่อผู้ใช้	50 ครั้ง/ นาที	หยุดการไล่อิงหลายบัญชี จากเครื่องเดียว

ปัญหาจริงที่เจอ และวิธีแก้

ตอนแรกระบบนับทุกครั้งที่เกิดเข้าสู่ระบบโดยดูจากหมายเลขเครื่องอย่างเดียว ผลคือทั้งวิทยาลัยใช้อินเทอร์เน็ตเส้นเดียวกัน จึงมีหมายเลขเครื่องเดียวกันหมด นักศึกษาคนที่ 6 ที่เข้าสู่ระบบในช่วงพักจึงถูกปฏิเสธทั้งที่พิมพ์รหัสถูก

จึงแก้เป็นสองอย่าง คือนับเฉพาะครั้งที่ผิด และแยกนับตามชื่อผู้ใช้งาน ทำให้การใช้งานปกติไม่ถูกระทบ ในขณะที่การเดารหัสของบัญชีหนึ่งยังถูกจำกัดที่ 5 ครั้งต่อนาทีเหมือนเดิม

6.8 ส่วนหัวความปลอดภัยของ HTTP

ไฟล์ `public/.htaccess` สั่งให้เซิร์ฟเวอร์แนบส่วนหัวเหล่านี้ไปกับทุกหน้า

ส่วนหัว	ทำหน้าที่อะไร
<code>Content-Security-Policy</code>	อนุญาตให้โหลดสคริปต์ ฟอนต์ และรูปได้จากเว็บของเราเองเท่านั้น ถ้ามีใครแทรกแท็กที่ชี้ไปเว็บอื่น เบราวเซอร์จะไม่โหลดให้
<code>X-Frame-Options: DENY</code>	ห้ามเว็บอื่นนำหน้าของเราไปซ้อนไว้ในกรอบ ป้องกันการหลอกให้เจ้าหน้าที่กดปุ่มโดยไม่รู้ตัว
<code>X-Content-Type-Options: nosniff</code>	ห้ามเบราวเซอร์เดาชนิดไฟล์เอง ป้องกันไฟล์ข้อความถูกตีความเป็นสคริปต์
<code>Referrer-Policy</code>	ไม่ส่งที่อยู่หน้าเต็มๆ ไปให้เว็บอื่น เพราะที่อยู่ของหน้ารายงานมีรหัสนักศึกษาติดอยู่
<code>Permissions-Policy</code>	ปิดการขอใช้กล้อง ไมโครโฟน และตำแหน่งที่ตั้ง เพราะระบบไม่ได้ใช้

นอกจากนี้ยังมีกฎห้ามเสิร์ฟไฟล์ที่ขึ้นต้นด้วยจุดทุกไฟล์ เพื่อกันกรณีที่วันหนึ่งมีคนวางไฟล์ `.env` ผิดที่

6.9 ไฟล์ .env เก็บอะไร

`.env` คือไฟล์ข้อความธรรมดาที่เก็บค่าซึ่งต่างกันไปในแต่ละเครื่องและเป็นความลับ จึงแยกออกมาจากโค้ด

ตัวแปร	ค่าเริ่มต้น	ความหมาย
<code>DB_HOST</code>	<code>localhost</code>	ที่อยู่เครื่องฐานข้อมูล
<code>DB_USER</code>	<code>root</code>	ชื่อผู้ใช้ฐานข้อมูล
<code>DB_PASSWORD</code>	(ว่างในเครื่องพัฒนา)	รหัสผ่านฐานข้อมูล ซึ่งเป็นค่าที่ต้องเป็นความลับที่สุดในไฟล์นี้
<code>DB_NAME</code>	<code>library_checkin</code>	ชื่อฐานข้อมูล
<code>APP_ENV</code>	<code>dev</code>	<code>dev</code> หรือ <code>prod</code> ควบคุมความเข้มของคุกกี้ เช่น จะบังคับ HTTPS หรือไม่
<code>APP_TIMEZONE</code>	<code>Asia/Bangkok</code>	เขตเวลา ต้องตรงกับเวลาที่ฐานข้อมูลใช้
<code>LIBRARY_CLOSING_TIME</code>	<code>17:00</code>	เวลาปิดห้องสมุด ใช้คำนวณการปิดการเข้าใช้อัตโนมัติ
<code>LIBRARY_OPEN_DAYS</code>	<code>1,2,3,4,5</code>	วันที่เปิดทำการ จันทร์ = 1 ถึง อาทิตย์ = 7
<code>CHECKIN_COOLDOWN_SECONDS</code>	<code>10</code>	เวลาขั้นต่ำระหว่างการกดเช็คอินสองครั้งของคนเดียวกัน

ทำไมต้องแยกออกมาเป็นไฟล์ต่างหาก และทำไมห้ามเก็บลง Git

1. ค่าเหล่านี้ต่างกันทุกเครื่อง เครื่องที่ใช้พัฒนาต่อฐานข้อมูลด้วยผู้ใช้ `root` ที่ไม่มีรหัสผ่าน ส่วนเซิร์ฟเวอร์จริงใช้ชื่อผู้ใช้และรหัสผ่านคนละชุด ถ้าเขียนค่าเหล่านี้ลงในโค้ดตรงๆ จะต้องแก้ไฟล์ทุกครั้งที่นำขึ้นเซิร์ฟเวอร์ ซึ่งพลาดง่ายมาก
2. รหัสผ่านต้องไม่อยู่ในที่ที่คนอื่นเห็น โค้ดของโครงการนี้เปิดสาธารณะบน GitHub ถ้ารหัสผ่านฐานข้อมูลอยู่ในโค้ด ใครก็ตามที่เปิดดูจะเห็นทันที ไฟล์ `.env` จึงถูกระบุไว้ใน `.gitignore` และไม่เคยถูกบันทึกลงประวัติของ Git แม้แต่ครั้งเดียว
3. ถ้าหลุดต้องเปลี่ยนได้ทันที เมื่อค่าอยู่ในไฟล์เดียว การเปลี่ยนรหัสผ่านคือแก้ไฟล์นั้นไฟล์เดียว ไม่ต้องไล่แก้ทั้งโปรเจก

ในโปรเจกจะมีไฟล์ `.env.example` ที่มีเฉพาะชื่อตัวแปรแต่ไม่มีค่าจริง เก็บลง Git ได้ เพื่อให้คนที่มารับช่วงต่อรู้ว่าต้องกรอกอะไรบ้าง

6.10 การวางไฟล์ไว้นอกพื้นที่ที่เว็บเข้าถึงได้

บนเซิร์ฟเวอร์จริง โฟลเดอร์ที่เว็บเปิดให้เข้าถึงคือ `public_html` เท่านั้น ส่วนโฟลเดอร์ `src/` และไฟล์ `.env` ถูกวางไว้ชั้นบนของ `public_html` จึงไม่มีที่อยู่เว็บใดที่เข้าถึงได้เลย ต่อให้พิมพ์ `ntclibrary.com/.env` ก็จะได้หน้าไม่พบไฟล์

6.11 ข้อจำกัดที่ยังเหลืออยู่

เพื่อความตรงไปตรงมา ขอรบรูสิ่งที่ยังทำได้ไม่สมบูรณ์ไว้ด้วย

ข้อจำกัด	ความเสี่ยงที่ยังเหลือ	สิ่งที่ลดความเสี่ยงไว้แล้ว
ยังไม่มีโทเคน CSRF แยกต่อคำขอ	เว็บอื่นอาจหลอกให้ เบราว์เซอร์ของผู้ที่เข้าสู่ ระบบอยู่ส่งคำขอมาที่ ระบบ	คุณก็ตั้ง <code>SameSite=Lax</code> เบราว์เซอร์จึงไม่แนบคุกกี้ ไปกับคำขอที่เว็บอื่นเป็น คนส่ง และ API ทุกเส้น ทางรับเฉพาะ JSON
การนำไฟล์ขึ้น เซิร์ฟเวอร์ใช้ FTP ซึ่งไม่เข้ารหัส	ผู้ที่ดักข้อมูลระหว่าง ทางอาจเห็นรหัสผ่าน FTP	รหัสผ่านถูกเก็บเป็นความ ลับใน GitHub และไม่เคย ถูกพิมพ์ออกมา แผนถัด ไปคือเปลี่ยนไปใช้ FTPS
นโยบายความ ปลอดภัยของเนื้อหา ยังอนุญาตสคริปต์ใน บรรทัด	หากมีช่องโหว่ XSS นโยบายนี้จะไม่ช่วย หยุด	ตัวที่ป้องกันจริงคือการ แปลงอักขระก่อนแสดง ผลทุกจุด แผนถัดไปคือใช้ ค่าสุ่มต่อคำขอแทน

บทที่ 7 พีเจอร์และหน้าเว็บทั้งหมด

7.1 หน้าของนักศึกษา

7.1.1 หน้าเข้าสู่ระบบ

- ช่องเดียวสำหรับทุกคน ไม่ต้องเลือกก่อนว่าเป็นนักศึกษาหรือเจ้าหน้าที่ เพราะระบบดูจากคอลัมน์ `role` ของบัญชีนั้นเองแล้วส่งไปหน้าที่ถูกต้อง
- ปุ่มรูปดวงตาสำหรับเปิดดูรหัสผ่านที่พิมพ์ไป
- ปุ่มสลับธีมสว่าง-มืด และโหมดอัตโนมัติที่เปลี่ยนตามเวลาของวัน

7.1.2 หน้าสมัครสมาชิก

- กรอกรหัสนักศึกษา คำนำหน้า เพศ ชื่อ นามสกุล แผนกวิชา ระดับชั้น ชั้นปี และรหัสผ่าน
- ตรวจสอบความถูกต้องสองชั้น คือฝั่งหน้าเว็บเพื่อความสะดวก และฝั่งเซิร์ฟเวอร์เป็นตัวตัดสินจริง เพราะการตรวจที่หน้าเว็บอย่างเดียวข้ามได้
- สมัครเสร็จได้สถานะ `approved` ทันที เข้าสู่ระบบได้เลย

7.1.3 หน้าหลัก — ปุ่มเช็คอิน

- ปุ่มวงกลมใหญ่กลางจอปุ่มเดียว ทำหน้าที่สลับระหว่างเช็คอินกับเช็คเอาท์เอง โดยดูจากเหตุการณ์ล่าสุดของผู้ใช้คนนั้น สีเขียวคือเข้า สีแดงคือออก
- เมื่อกดเช็คอิน จะมีหน้าต่างให้เลือกเวลาที่ตั้งใจจะออก โดยเลือกได้สองแบบคือหมุนวงล้อเลือกเวลา หรือกดปุ่มจำนวนชั่วโมงสำเร็จรูป และมีทางลัด “จนกว่าจะปิด”
- ป้ายสถานะ 3 แบบ ได้แก่ กำลังอยู่ในห้องสมุด (สีเขียวกะพริบ) เช็คเอาท์แล้ววันนี้พร้อมบอกเวลาที่ออก (สีน้ำเงิน) และยังไม่ได้

เช็คอินวันนี้ (สี่สัปดาห์)

- นาฬิกาจับเวลาที่อยู่ในห้องสมุด เดินทุกวินาที
- แถบเตือนเมื่อใกล้ถึงเวลาที่ตั้งไว้ พร้อมปุ่มขอเวลาเพิ่ม 30 หรือ 60 นาที
- กล้องประกาศจากเจ้าหน้าที่ แสดงเมื่อมีประกาศที่เปิดใช้งานอยู่
- วันเสาร์-อาทิตย์ปุ่มจะถูกปิดพร้อมข้อความบอกวันเปิดทำการ
- หลังบันทึกสำเร็จ ปุ่มจะถูกปิด 10 วินาทีพร้อมนับถอยหลัง เพื่อป้องกันการกดซ้ำ

7.1.4 หน้าต่างประวัติการเข้าใช้

- แสดงเป็นรายการ “การมา 1 ครั้ง” ต่อ 1 แถว มีเวลาเข้า เวลาออก และระยะเวลาที่อยู่
- เลือกช่วงเวลาได้จากรายการเดียว คือ ทั้งหมด / 7 วันล่าสุด / 30 วันล่าสุด / เดือนนี้ / กำหนดเอง
- มีแถบสรุปของช่วงที่เลือก ได้แก่ จำนวนครั้ง เวลารวม และเฉลี่ยต่อครั้ง
- ระยะเวลาที่ต่ำกว่า 1 นาทีจะบอกเป็นวินาที เพื่อให้แยกออกจาก 3 วินาทีกับ 55 วินาทีต่างกัน
- ถ้าเป็นการปิดโดยระบบหรือโดยเจ้าหน้าที่ จะมีหมายเหตุกำกับไว้ว่าใครเป็นคนปิดให้

7.1.5 หน้าข้อมูลผู้ใช้

- ดูและแก้ไขข้อมูลส่วนตัวของตนเอง
- เปลี่ยนรหัสผ่าน โดยต้องกรอกรหัสผ่านปัจจุบันเพื่อยืนยันตัวตนก่อน

7.2 หน้าของเจ้าหน้าที่

7.2.1 หน้าภาพรวม

- กล่องตัวเลขสรุป 4 กล่อง คือ จำนวนครั้งที่เข้าใช้ จำนวนนักศึกษาที่เข้าใช้ เฉลี่ยการเข้าใช้ต่อวัน และผู้ที่อยู่ในห้องสมุดขณะนี้ แต่ละกล่องมีปุ่มเครื่องหมายคำถามให้กดดูคำอธิบาย
- ตัวกรองช่วงเวลา คือ เดือนนี้ / 7 วันล่าสุด / วันนี้
- กราฟแนวโน้มรายวัน กดที่แท่งเพื่อดูรายละเอียดของวันนั้นแยกตามแผนก แล้วเจาะลงไปถึงรายชื่อได้
- อันดับแผนกที่เข้าใช้มากที่สุด กราฟรายชั่วโมง และกราฟวันในสัปดาห์
- ช่องพิมพ์ประกาศถึงนักศึกษา พร้อมสวิตช์เปิด-ปิด และตัวนับ 500 ตัวอักษร
- ตัวเลขผู้ที่อยู่ในห้องสมุดขณะนี้ถูกดึงใหม่ทุก 20 วินาที

7.2.2 หน้าทำเนียบสมาชิก

- ค้นหาด้วยรหัสหรือชื่อ กรองด้วยแผนกวิชา ระดับชั้น ชั้นปี และสถานะบัญชี
- เรียงตามรหัสนักศึกษา ไม่ใช่เรียงตามชื่อ เพราะเจ้าหน้าที่ค้นจากรหัสที่อยู่ในเอกสารตรงหน้า
- แก้ไขข้อมูลสมาชิก เปลี่ยนสถานะบัญชี เปลี่ยนสิทธิ์ และลบสมาชิก
- รีเซ็ตรหัสผ่านโดยระบบส่งรหัสชั่วคราวมาให้
- แบ่งหน้า ครั้งละ 10 รายการ

7.2.3 หน้าผู้ที่กำลังใช้งานอยู่

แสดงรายชื่อผู้ที่เหตุการณ์ล่าสุดเป็น “เข้า” พร้อมเวลาที่เช็คอินและเวลาที่ตั้งใจจะออก และมีปุ่มบังคับเช็คเอาท์สำหรับกรณีที่นักศึกษา

ออกไปแล้วแต่ลืมกด

7.2.4 หน้าประวัติการเช็คชื่อทั้งหมด

ดูประวัติของทุกคนย้อนหลัง พร้อมตัวกรองและการแบ่งหน้า

7.3 กลไกอัตโนมัติที่ทำงานเบื้องหลัง

กลไก	ทำอะไร
ปิดการเข้าใช้ที่ค้างอยู่	ปิดให้อัตโนมัติเมื่อเลยเวลาที่ตั้งใจจะออก หรือเลยเวลาปิดห้องสมุดของวันที่เช็คอิน โดยบันทึกที่มาไว้ว่าเป็น <code>auto</code> เพื่อให้แยกออกจากการกดเองได้
ระงับบัญชีที่หมดอายุ	เปลี่ยนสถานะเป็น <code>retired</code> เมื่อบัญชีมีอายุครบ 1 ปีการศึกษา นับจากวันที่สร้าง
ล้างประวัติการกรอกรหัสผิด	ลบแถวที่เก่ากว่า 1 นาทีถึงทุกครั้งที่มีการตรวจตารางจึงไม่โตขึ้นเรื่อยๆ

ทั้งสามอย่างทำงานแทรกอยู่ในทุกคำขอที่เข้ามา ไม่ได้ตั้งเป็นงานตามเวลา (cron) เหตุผลคือโฮสต์ที่ใช้จำกัดจำนวนงานตามเวลา และปริมาณการใช้งานระดับวิทยาลัยหนึ่งแห่งทำให้ต้นทุนของการตรวจแบบนี้ต่ำมาก

บทที่ 8 ระบบรายงาน

8.1 รายงานทั้ง 8 แบบ

รายงาน	ตอบคำถามอะไร
แบบแดชบอร์ด	ภาพรวมของช่วงเวลาหนึ่ง มีตัวเลขสรุป กราฟรายวัน กราฟรายชั่วโมง สัดส่วนเพศ และตารางแยกตามแผนก
รายวัน	ในวันหนึ่งมีใครเข้าใช้บ้าง เวลาเท่าไร
รายเดือน	ในเดือนหนึ่งแต่ละคนเข้าใช้กี่ครั้ง
แยกตามแผนกวิชา	แผนกใดใช้บริการมากน้อยเพียงใด
สรุปผู้บริหาร	ตัวเลขสำคัญพร้อมการเปรียบเทียบกับเดือนก่อน
เปรียบเทียบ	เทียบสองช่วงเวลาว่าต่างกันอย่างไร
รายภาคเรียน	ภาพรวมทั้งภาคเรียน
ค้นรายบุคคล	นักศึกษาคนหนึ่งเข้าใช้เมื่อไรบ้าง

8.2 ตัวกรองร่วมของทุกรายงาน

ทุกรายงานใช้ตัวกรองชุดเดียวกัน คือช่วงเวลา แผนกวิชา ระดับชั้น ภาคเรียน และปีการศึกษา และที่สำคัญคือทุกรายงานเรียกใช้ฟังก์ชันคำนวณตัวเดียวกันที่เก็บไว้ในไฟล์ `aggregate.php` จึงเป็นไปได้ที่รายงานสองแบบจะให้ตัวเลขไม่ตรงกันสำหรับเงื่อนไขเดียวกัน ซึ่งเป็นสิ่งที่ทำให้รายงานเชื่อถือไม่ได้ทั้งระบบถ้าเกิดขึ้น

8.3 การส่งออกไฟล์ และวิธีทำกราฟลงใน PDF

รูปแบบ	สร้างด้วย	เหมาะกับ
PDF	mPDF ที่ฝั่งเซิร์ฟเวอร์	แนบเอกสารราชการ ส่งให้ผู้บริหาร หรือ สัมภาษณ์

รูปแบบ	สร้างด้วย	เหมาะกับ
CSV	PHP เขียนไฟล์ เอง	เปิดต่อในโปรแกรมตารางคำนวณ
Excel	PHP เขียนไฟล์ เอง	นำไปคำนวณต่อ

ปัญหาทางเทคนิคที่เจอจริง และวิธีแก้

กราฟบนหน้าเว็บสร้างจาก `<div>` ที่กำหนดความสูงเป็นเปอร์เซ็นต์ ซึ่งเป็นเทคนิคที่เบราว์เซอร์รองรับดี แต่ mPDF ซึ่งมีตัวแปล CSS เป็นของตัวเองคำนวณความสูงแบบนี้ผิด ผลคือไฟล์ PDF ที่ได้มีหน้าว่างพ่วงมา 50 ถึง 300 หน้า

วิธีแก้ คือให้ไลบรารี GD ซึ่งมากับ PHP อยู่แล้ว วาดกราฟออกมาเป็นรูปภาพ PNG ที่เสร็จสมบูรณ์ก่อน แล้ว mPDF มีหน้าที่เพียงวางรูปลงไป ซึ่งเป็นงานที่มันทำได้แน่นอน ปัจจุบันระบบวาดทั้งกราฟแท่งแนวตั้ง กราฟแท่งแนวนอน และกราฟวงกลมแบบโดนัทด้วยวิธีนี้

รายละเอียดเล็กน้อยของการวาดกราฟวงกลมคือ ฟังก์ชันวาดส่วนโค้งของ GD ไม่มีการลบรอยหยัก ระบบจึงวาดที่ขนาดใหญ่กว่าจริง 4 เท่าแล้วย่อลงเพื่อให้ขอบวงเรียบ

บทที่ 9 การนำระบบขึ้นใช้งานจริง

9.1 Hostinger และโครงไฟล์บนเซิร์ฟเวอร์

Hostinger คือผู้ให้บริการพื้นที่เว็บแบบแบ่งปันทรัพยากร (shared hosting) หมายความว่าเครื่องเซิร์ฟเวอร์หนึ่งเครื่องที่หลายเว็บใช้ร่วมกัน โดยแต่ละเว็บถูกกั้นพื้นที่ของตัวเองไว้ สิ่งที่ได้มาพร้อมกับบริการคือ

- PHP และ MariaDB ติดตั้งไว้ให้แล้ว ไม่ต้องตั้งค่าเซิร์ฟเวอร์เอง
- ชื่อโดเมน ntclibrary.com
- ใบรับรอง HTTPS ทำให้ข้อมูลระหว่างเบราว์เซอร์กับเซิร์ฟเวอร์ถูกเข้ารหัส
- บัญชี FTP สำหรับอัปโหลดไฟล์
- หน้าจัดการฐานข้อมูล phpMyAdmin

โครงไฟล์บนเซิร์ฟเวอร์จริงถูกจัดวางดังนี้

```

/ (พื้นที่ของบัญชีนี้)
|
├── .env          ← รหัสฐานข้อมูล - อยู่นอก public_html เข้าจากเว็บไม่ได้
|               |
|   ├── vendor/   ← ไลบรารีภายนอก (mPDF)
|   |   ├── fonts/ ← ฟอนต์สำหรับสร้าง PDF
|   |   └── src/   ← โค้ดหลัก - อยู่นอก public_html เข้าจากเว็บไม่ได้
|   |               |
|   |   ├── public_html/ ← โฟลเดอร์เดียวที่เว็บเปิดให้เข้าถึง
|   |   |   ├── index.php
|   |   |   ├── login.php  dashboard.php  admin-*.php
|   |   |   ├── partials/
|   |   |   └── assets/

```

จุดสำคัญคือ `public_html` ตรงกับโฟลเดอร์ `public` ในโปรเจกต์ ส่วนโฟลเดอร์อื่นทั้งหมดอยู่ชั้นบนขึ้นไปหนึ่งชั้น จึงไม่มีที่อยู่เว็บใดชี้ถึงได้

9.2 Git และ GitHub ใช้ทำอะไรบ้าง

Git คือระบบเก็บประวัติการแก้ไขไฟล์ ส่วน GitHub คือบริการที่เก็บสำเนาของ Git ไว้บนอินเทอร์เน็ต ทั้งสองอย่างทำหน้าที่ต่างกัน ดังนี้

หน้าที่	อธิบาย
1. เก็บประวัติทุกการแก้ไข	ทุกครั้งที่แก้ไขโค้ดเสร็จจะบันทึกเป็นหนึ่ง commit พร้อมข้อความอธิบายว่าแก้ไขอะไรและทำไม ทำให้ย้อนดูได้ว่าโค้ดบรรทัดหนึ่งถูกเขียนขึ้นเพราะเหตุใด
2. ย้อนกลับได้เมื่อแก้แล้วพัง	ถ้าแก้แล้วระบบทำงานผิด สามารถย้อนกลับไปสถานะก่อนหน้าได้ทันที ไม่ต้องนั่งไล่แก้ตัวเอง
3. เป็นสำเนาสำรองนอกเครื่อง	ถ้าคอมพิวเตอร์ที่ใช้พัฒนาเสีย โค้ดทั้งหมดยังอยู่ครบบน GitHub
4. เป็นต้นทางของการนำขึ้นเซิร์ฟเวอร์	ระบบนำขึ้นอัตโนมัติอ่านจาก GitHub ว่ามีไฟล์ใดเปลี่ยนไปบ้างตั้งแต่ครั้งที่แล้ว แล้วอัปเดตเฉพาะไฟล์เหล่านั้น
5. กันไฟล์ที่ไม่ควรขึ้นไป	ไฟล์ <code>.env</code> ถูกระบุไว้ใน <code>.gitignore</code> จึงไม่ถูกบันทึกลงประวัติ ทำให้รหัสผ่านฐานข้อมูลไม่เคยไปอยู่บน GitHub

9.3 ขั้นตอนการนำขึ้นอัตโนมัติ

ระบบใช้ GitHub Actions ซึ่งเป็นบริการรันคำสั่งอัตโนมัติของ GitHub โดยขั้นตอนทั้งหมดเป็นดังนี้

1. แก้โค้ดในเครื่อง แล้วทดสอบให้ผ่าน
- ▼
2. บันทึกเป็น commit แล้วส่งขึ้น GitHub
- ▼
3. สั่งให้ระบบนำขึ้นทำงาน โดยกรอก
 - ซ่อมก่อนหรือไม่ (รอบแรกต้องเป็น "ใช่" เสมอ)
 - พิมพ์คำว่า deploy เพื่อยืนยัน
 - ระบุว่าจะอัปเดตไฟล์ที่เปลี่ยนตั้งแต่ commit ไหน
- ▼
4. ระบบเทียบว่ามีไฟล์ใดเปลี่ยนบ้าง แล้วแสดงรายชื่อออกมาโดยยังไม่เขียนอะไรจริง
- ▼
5. ตรวจสอบรายชื่อว่าครบและไม่มีไฟล์แปลกปลอม
- ▼
6. สั่งอีกครั้งโดยเปลี่ยนเป็นอัปเดตจริง – ไฟล์ถูกส่งขึ้นตามลำดับที่กำหนดไว้
- ▼
7. ระบบตรวจให้อัตโนมัติหลังอัปเดตเสร็จ
 - เปิด /login ต้องได้ 200 (เว็บยังทำงาน)
 - เปิด /src/auth.php ต้องได้ 404 (ไฟล์อยู่ถูกชั้น)

รหัสผ่าน FTP ไม่ได้อยู่ในโค้ด แต่เก็บไว้ในระบบเก็บความลับของ GitHub และถูกใส่ให้เฉพาะตอนที่คำสั่งนำขึ้นทำงานเท่านั้น ผู้ที่เปิดดูโค้ดจึงไม่เห็นค่านี้

9.4 เหตุผลที่ต้องซั่มก่อนทุกครั้ง

บทเรียนจากเหตุการณ์จริง

ครั้งหนึ่งระบบนำขึ้นตั้งค่าให้ดึงประวัติจาก Git มาเพียง 1 commit ล่าสุด แล้วเทียบว่ามีไฟล์ใดเปลี่ยนไปบ้าง ผลคือเมื่องานหนึ่งรอบถูกแบ่งเป็นหลาย commit ตามเรื่อง ระบบจะเห็นเฉพาะไฟล์ของ commit สุดท้าย และอัปเดตไปเพียงไม่กี่ไฟล์โดยไม่แจ้งเตือนอะไรเลย เว็บจึงอยู่ในสภาพที่โค้ดใหม่กับโค้ดเก่าปนกัน

วิธีแก้คือให้ดึงประวัติทั้งหมด และเพิ่มช่องให้ระบุเองว่าจะเทียบตั้งแต่ commit ไหน พร้อมกำหนดว่าถ้าหา commit นั้นไม่เจอให้หยุดทันที แทนที่จะอัปเดตไปเรื่อยๆ เพราะการหยุดแล้วแจ้งเตือนดีกว่าการทำงานครั้งเดียวโดยไม่มีใครรู้

บทที่ 10 การทดสอบระบบ

10.1 ชุดทดสอบอัตโนมัติ

ระบบมีชุดทดสอบตั้งแต่ต้นจนจบจำนวน 27 กรณี เขียนด้วย PHP ล้วนโดยไม่ใช้เครื่องมือทดสอบภายนอก และทดสอบกับฐานข้อมูลจริงไม่ใช่ข้อมูลจำลอง โดยชุดทดสอบจะสร้างบัญชีชั่วคราวของตัวเองขึ้นมาแล้วลบทิ้งทุกครั้งเมื่อจบ

สิ่งที่ครอบคลุมได้แก่ สมัครสมาชิก เข้าสู่ระบบ เช็คอิน กดซื้อทันทีแล้วต้องถูกปฏิเสธ เช็คเอาต์ ดูประวัติ เปลี่ยนรหัสผ่าน เข้าสู่ระบบด้วยรหัสเก่าแล้วต้องไม่ผ่าน การกั้นไม่ให้นักศึกษาเข้าถึงหน้าเจ้าหน้าที่ และการเปิดรายงานทุกแบบ

10.2 การตรวจไวยากรณ์

```
php -l <ไฟล์>      ตรวจไฟล์ PHP
node --check <ไฟล์> ตรวจไฟล์ JavaScript
```

10.3 การตรวจด้วยสายตาบนอุปกรณ์จริง

เนื่องจากผู้ใช้ส่วนใหญ่เปิดผ่านโทรศัพท์ ทุกหน้าจึงถูกตรวจที่ความกว้างหน้าจอ 390 พิกเซลซึ่งเป็นขนาดของโทรศัพท์ทั่วไป โดยวัดว่าไม่มีหน้าใดล้นออกด้านข้างจนต้องเลื่อนซ้ายขวา

10.4 ข้อควรระวังของชุดทดสอบ

ผลทดสอบที่ “ไม่ผ่าน” บางกรณีคือพฤติกรรมที่ถูกต้อง

ชุดทดสอบผูกกับเวลาจริงและวันจริง มี 5 กรณีที่ต้องเช็คอิน ซึ่งจะไม่ผ่านถ้ารันหลังเวลาปิดห้องสมุด หรือรันในวันเสาร์-อาทิตย์ เพราะระบบปฏิเสธการเช็คอินตามกฎหมายที่ออกแบบไว้

วิธีรันนอกเวลาทำการคือกำหนดค่าชั่วคราวตอนเปิดเซิร์ฟเวอร์ทดสอบ

```
LIBRARY_CLOSING_TIME=23:30  
LIBRARY_OPEN_DAYS=1,2,3,4,5,6,7 \  
php -S localhost:8000 -t public
```

บทที่ 11 แนวคำถาม–คำตอบสำหรับการนำเสนอ

รวบรวมคำถามที่มีโอกาสถูกถามมากที่สุด พร้อมคำตอบที่อ้างอิงจากสิ่งที่ระบบทำจริง

กลุ่มที่ 1 — ฐานข้อมูล

1. แอดมินกับผู้ใช้แยกตารางกันไหม

ไม่แยกครับ อยู่ในตาราง `users` ตารางเดียวกัน แล้วใช้คอลัมน์ `role` ซึ่งเป็นชนิด ENUM รับได้แค่สองค่าคือ `student` กับ `admin` เป็นตัวแยก

เหตุผลหลักคือทั้งสองกลุ่มมีสิ่งที่เหมือนกันคือ “การเป็นบัญชีที่เข้าสู่ระบบได้” ต่างต้องมีชื่อผู้ใช้ รหัสผ่านที่เข้ารหัสแล้ว และสถานะบัญชี ถ้าแยกสองตารางจะต้องมีคอลัมน์ชุดเดียวกันนี้ซ้ำกัน และตอนเข้าสู่ระบบต้องไล่ค้นทีละตาราง อีกทั้งตาราง `checkin_logs` ก็จะไม่รู้ว่าเป็นใครนอกควรชี้ไปที่ตารางไหน

สิ่งที่แยกออกไปจริงคือข้อมูลส่วนบุคคล ซึ่งอยู่ในตาราง `students` เพราะข้อมูลอย่างแผนกวิชา ชั้นปี และห้องเรียน มีเฉพาะนักศึกษาเท่านั้น เจ้าหน้าที่ไม่มี ถ้ายัดรวมไว้ที่เดียวกัน แถวของเจ้าหน้าที่จะมีคอลัมน์ว่างติดอยู่ตลอด

2. ไม่มีตัวกรองหรือว่าอันไหนเป็น user อันไหนเป็นแอดมิน

มีครับ คือคอลัมน์ `role` และมีการนำไปใช้ตรวจอยู่ 3 จุด ทั้งหมดอยู่ที่ฝั่งเซิร์ฟเวอร์

จุดที่ 1 ตอนเข้าสู่ระบบสำเร็จ ระบบอ่านค่า `role` จากฐานข้อมูลมาเก็บไว้ในเซสชันที่เซิร์ฟเวอร์ ผู้ใช้แก้ค่านี้จากเบราว์เซอร์ไม่ได้

จุดที่ 2 ทุกฟังก์ชันที่เป็นงานของเจ้าหน้าที่จะเรียก `require_admin()` เป็นบรรทัดแรก ถ้าค่าในเซสชันไม่ใช่ `admin` จะตอบรหัส 403 ทันทีโดยไม่ทำงานต่อ

จุดที่ 3 ทุกหน้าเว็บของเจ้าหน้าที่เรียกไฟล์ `guard.php` เป็นบรรทัดแรกสุด ถ้าเป็นนักศึกษาจะถูกส่งกลับไปหน้าหลักก่อนที่หน้าเว็บจะถูกแสดงแม้แต่บรรทัดเดียว

ในหน้าเว็บมีการซ่อนเมนูตามสิทธิ์ด้วย แต่นั่นเป็นแค่ความสะดวก ไม่ใช่การป้องกัน เพราะสิ่งที่ซ่อนด้วย JavaScript ผู้ใช้เอากลับมาได้เสมอ

3. ทำไม `checkin_logs` ถึงเก็บเข้ากับออกเป็นคนละแถว ไม่เก็บแถวเดียวที่มีทั้งสองเวลา

เพราะตอนที่นักศึกษาเช็คอิน ระบบยังไม่รู้ว่าเขาจะออกเมื่อไร ถ้าเก็บแถวเดียว จะต้องสร้างแถวที่มีคอลัมน์เวลาออกกว้างไว้ก่อนแล้วกลับมาแก้ทีหลัง ซึ่งแปลว่าทุกคำสั่งที่อ่านตารางนี้ต้องเผื่อกรณีแถวที่ยังไม่สมบูรณ์เสมอ

การเก็บเป็นเหตุการณ์ทำให้ทุกแถวสมบูรณ์ตั้งแต่วินาทีที่เขียน และไม่มีการแก้ไขแถวเดิมเลย มีแต่การเพิ่มแถวใหม่ เวลาต้องแสดงเป็น “การมา 1 ครั้ง” ค่อยใช้ฟังก์ชัน `LEAD()` ของ SQL จับคู่แถวเข้ากับแถวออกให้

4. ทำไม `users.student_id` ไม่ตั้งเป็น **UNIQUE**

ในทางปฏิบัติใช้แบบหนึ่งคนหนึ่งบัญชี แต่ไม่ได้บังคับไว้ในระดับฐานข้อมูล ความสัมพันธ์จึงเป็นแบบหนึ่งต่อกลุ่มในทางเทคนิค ข้อดีคือถ้าอนาคตต้องให้นักศึกษาคนหนึ่งมีบัญชีเจ้าหน้าที่เพิ่มขณะช่วยงานห้องสมุด ก็ทำได้โดยไม่ต้องแก้โครงสร้าง

5. ทำไมต้องสร้างดัชนี (index) เอง ในเมื่อมีคีย์นอกอยู่แล้ว

คีย์นอกของ MariaDB สร้างดัชนีให้เฉพาะคอลัมน์ที่เป็นคีย์นอกเท่านั้น คือ `user_id` แต่รายงานทุกแบบกรองด้วย `ช่วงวันที่` ซึ่งไม่มีดัชนี ฐานข้อมูลจึงต้องไล่อ่านทั้งตารางทุกครั้ง จึงเพิ่มดัชนีบน `timestamp` และดัชนีผสม `(user_id, timestamp)` สำหรับการค้นประวัติรายคนเรียงจากใหม่ไปเก่า

กลุ่มที่ 2 — ความปลอดภัย

6. รหัสผ่านเก็บยังไง เข้ารหัสด้วยอะไร

ใช้ **bcrypt** ผ่านฟังก์ชัน `password_hash()` ของ PHP ครับ และขอเรียกให้ถูกว่าเป็นการ **hash** ไม่ใช่การ **encrypt** เพราะการเข้ารหัสแบบ **encrypt** ถอดกลับได้ถ้ามีกุญแจ แต่ **hash** ถอดกลับไม่ได้เลย

สิ่งที่เก็บในฐานข้อมูลจึงไม่ใช่รหัสผ่าน แต่เป็นผลลัพธ์ของการแปลงเวลาตรวจสอบเข้าสู่ระบบ ระบบจะนำรหัสที่ผู้ใช้เพิ่งพิมพ์มาแปลงด้วยเกลือชุดเดิม แล้วเทียบว่าผลลัพธ์ตรงกันหรือไม่ ไม่ได้ถอดค่าเดิมออกมาดู

7. ในฐานะข้อมูล คนที่เปิดดูจะเห็นรหัสผ่านเป็นแบบไหน

เห็นเป็นข้อความยาว 60 ตัวอักษรแบบนี้ครับ

```
$2y$10$R5mw3n3SWxa9H2jvufa7n.A6rY6y2CkRS1L5k1JFcf1auNRjK.oJK
```

อ่านได้ว่า `$2y$` คืออัลกอริทึม bcrypt, `10` คือความหนักซึ่งหมายถึงจำนวนซ้ำ 1,024 รอบ, 22 ตัวถัดมาคือเกลือที่สุ่มใหม่ทุกบัญชี และที่เหลือคือค่าที่แปลงแล้ว

ผลของการมีเกลือคือ ต่อให้ผู้ใช้สองคนตั้งรหัสผ่านเหมือนกันเป๊ะ ค่าที่เก็บก็จะไม่เหมือนกัน คนที่ขโมยฐานข้อมูลไปจึงเดาไม่ได้ว่าใครใช้รหัสซ้ำกับใคร

8. ทำไมไม่ใช่ MD5

เพราะ MD5 ถูกออกแบบมาให้เร็ว ซึ่งดีสำหรับการตรวจความถูกต้องของไฟล์ แต่เป็นข้อเสียร้ายแรงสำหรับรหัสผ่าน การ์ดจอสมัยใหม่คำนวณ MD5 ได้หลายพันล้านค่าต่อวินาที การไล่เดารหัสผ่านทั่วไปจึงใช้เวลาไม่นาน

bcrypt ออกแบบมาให้ช้าโดยตั้งใจ และปรับให้ช้าลงอีกได้เมื่อคอมพิวเตอร์ในอนาคตเร็วขึ้น อีกทั้งใส่เกลือให้อัตโนมัติ ในขณะที่ MD5 ต้องจัดการเกลือเอง นอกจากนี้ MD5 ยังถูกพบวิธีสร้างข้อมูลสองชุดที่ให้ค่าเดียวกันแล้ว จึงไม่เหมาะกับงานความปลอดภัยใดๆ อีกต่อไป

9. ถ้านักศึกษาลืมรหัสผ่าน แอดมินดูให้ได้ไหม

ดูไม่ได้ครับ และนี่เป็นคุณสมบัติที่ตั้งใจ ไม่ใช่ข้อจำกัด เพราะระบบไม่ได้เก็บรหัสผ่านไว้ในรูปแบบที่อ่านได้ตั้งแต่แรก

วิธีที่ใช้คือเจ้าหน้าที่กดปุ่มรีเซ็ตรหัสผ่าน ระบบจะสุ่มรหัสชั่วคราวขึ้นมาใหม่ เข้ารหัสเก็บทับค่าเดิม แล้วแสดงรหัสชั่วคราวบนหน้าจอครั้งเดียวให้เจ้าหน้าที่บอกนักศึกษา โดยรหัสที่สุ่มจะตัดตัวอักษรที่สับสนออก เช่น เลข 0 กับตัว O และเลข 1 กับตัว l และ I เพราะต้องอ่านออกเสียงให้ฟังกัน

10. ป้องกัน SQL Injection ยังไง

ใช้คำสั่งแบบเตรียมไว้ล่วงหน้า (prepared statement) ของ PDO ทุกจุดที่ติดต่อฐานข้อมูล ไม่มีจุดใดในระบบที่เอาค่าจากผู้ใช้มาต่อเป็นข้อความในคำสั่ง SQL โดยตรง

หลักการคือส่ง**โครงสร้างคำสั่ง**ไปให้ฐานข้อมูลแปลก่อน แล้วจึงส่ง**ค่า**ตามไปที่หลังคนละรอบ ฐานข้อมูลจึงถือว่าสิ่งที่ผู้ใช้พิมพ์เป็นข้อมูลเสมอ ต่อให้พิมพ์คำสั่ง SQL ลงไปในช่องชื่อผู้ใช้อีกจะถูกค้นหาเป็นชื่อผู้ใช้จริงๆ ไม่ถูกนำไปทำงาน

และปิดโหมตจำลองคำสั่งไว้ด้วย เพื่อให้เป็นการเตรียมคำสั่งที่ฝั่งฐานข้อมูลจริง ไม่ใช่ให้ PHP ต่อข้อความให้แล้วส่งไปทั้งก้อน

11. ถ้ามีคนพยายามเดารหัสผ่านรั่วๆ ระบบทำยังไง

ทุกครั้งที่กรอกผิด ระบบบันทึกลงตาราง `login_attempts` แล้วนับย้อนหลัง 1 นาที ถ้าคู่ของเครื่องกับชื่อผู้ใช้เดียวกันผิดเกิน 5 ครั้ง หรือเครื่องเดียวกันผิดเกิน 50 ครั้งรวมทุกบัญชี จะถูกปฏิเสธด้วยรหัส 429 มีรายละเอียดที่เจอจากการใช้งานจริงคือ ตอนแรกนับทุกครั้งที่กด โดยดูจากหมายเลขเครื่องอย่างเดียว ผลคือทั้งวิทยาลัยออกอินเทอร์เน็ตเส้นเดียวกันจึงมีหมายเลขเดียวกันหมด นักศึกษาคนที่ 6 ที่เข้าสู่ระบบช่วงพักถูกปฏิเสธทั้งที่พิมพ์รหัสถูก จึงแก้เป็นนับเฉพาะครั้งที่ผิดและแยกนับตามชื่อผู้ใช้

12. ไฟล์ .env เก็บอะไร ทำไมต้องมี

เก็บค่าที่ต่างกันในแต่ละเครื่องและเป็นความลับ ได้แก่ ที่อยู่ ชื่อผู้ใช้ รหัสผ่าน และชื่อฐานข้อมูล โหมดการทำงาน (dev หรือ prod) เขตเวลา เวลาปิดห้องสมุด วันเปิดทำการ และเวลาขั้นต่ำระหว่างการกดเช็คอินสองครั้ง

เหตุผลที่ต้องแยกออกมามี 3 ข้อ ข้อแรกคือค่าเหล่านี้ต่างกันทุกเครื่อง ถ้าเขียนลงในโค้ดตรงๆ จะต้องแก้ไฟล์ทุกครั้งที้นำขึ้นเซิร์ฟเวอร์ ข้อสองคือโค้ดของโครงการเปิดสาธารณะบน GitHub ถ้าวัดผ่านฐานข้อมูลอยู่ในโค้ด ใครก็เห็น และข้อสามคือถ้าค่าหลุด การเปลี่ยนทำได้ทีไฟล์เดียว

ไฟล์นี้ถูกระบุใน `.gitignore` จึงไม่เคยถูกบันทึกลงประวัติของ Git แม้แต่ครั้งเดียว และบนเซิร์ฟเวอร์จริงยังถูกวางไว้นอกโฟลเดอร์ที่เว็บเข้าถึงได้อีกชั้นหนึ่ง

13. ทำไมต้องแยกโฟลเดอร์ public กับ src

เพราะ `public` เป็นโฟลเดอร์เดียวที่เว็บเปิดให้เข้าถึงจากภายนอก ส่วน `src` ซึ่งมีคำสั่งฐานข้อมูลและตรรกะการตรวจสอบสิทธิ์อยู่นอกพื้นที่นั้น

สมมติวันหนึ่งเซิร์ฟเวอร์ตั้งค่าผิดจนหยุดแปลไฟล์ PHP ถ้าไฟล์เหล่านั้นอยู่ในพื้นที่ที่เว็บเข้าถึงได้ คนที่พิมพ์ที่อยู่ตรงเข้าไปจะเห็นซอร์สโค้ดทั้งไฟล์เป็นข้อความธรรมดา รวมถึงเห็นวิธีที่ระบบตรวจสอบสิทธิ์ การวางคนละชั้นทำให้ต่อให้เกิดเหตุนั้นก็ยังไม่ถึง

และระบบนำขึ้นเซิร์ฟเวอร์จะตรวจเรื่องนี้ทุกครั้ง โดยลองเรียก `/src/auth.php` จากภายนอก ถ้าไม่ได้รับรหัส 404 จะแจ้งเตือนทันที

กลุ่มที่ 3 — การออกแบบระบบ

14. ทำไมต้องแยกหน้าของแอดมินกับของผู้ใช้ ทำไมไม่ใช้หน้าเดียวแล้วซ่อนปุ่ม

มี 4 เหตุผลครับ

หนึ่ง งานที่ทำคนละเรื่องกันโดยสิ้นเชิง นักศึกษาเปิดเว็บมาเพื่อกดปุ่มเดียว หน้าจอจึงควรมีปุ่มนั้นเด่นที่สุด ส่วนเจ้าหน้าที่ต้องการตาราง ตัวกรอง และกราฟ ถ้ารวมกันทั้งสองฝ่ายจะได้หน้าจอที่รกเกินความจำเป็น

สอง ลดโอกาสพลาดด้านความปลอดภัย หน้าทีรวมทุกอย่างแล้วซ่อนตามสิทธิ์ เสี่ยงที่วันหนึ่งจะลืมนำเงื่อนไขซ่อนกับปุ่มใหม่ที่เพิ่งเพิ่ม แต่เมื่อแยกไฟล์ หน้าแอดมินทุกไฟล์ตั้ง `$requireAdmin = true;` ไว้บรรทัดบนสุด ทั้งไฟล์จึงถูกป้องกันโดยอัตโนมัติ

สาม โหลดเร็วกว่าหน้านักศึกษาไม่ต้องโหลดโค้ดกราฟและตาราง ซึ่งสำคัญเพราะส่วนใหญ่เปิดผ่านโทรศัพท์

สี่ แก้ไขต่อได้ง่ายกว่า การแก้หน้าเจ้าหน้าที่ไม่มีทางทำให้ปุ่มเช็คอินพัง

ส่วนที่ใช้ร่วมกันได้ก็ยังใช้ร่วมกัน เช่น แถบบนสุดและไฟล์ CSS อยู่ในโฟลเดอร์ที่ทั้งสองฝั่งเรียกใช้ จึงไม่ได้เขียนซ้ำ

15. ทำไมไม่ใช้เฟรมเวิร์กอย่าง Laravel

ระบบมี 30 เส้นทาง ซึ่งเขียนเป็นตารางจับคู่ธรรมดาได้ครบในไฟล์เดียว เฟรมเวิร์กจะเพิ่มไฟล์และขั้นตอนติดตั้งที่ต้องดูแล โดยไม่ได้แก้ปัญหาอะไรที่มีอยู่จริงในโครงการนี้ อีกทั้งโฮสต์แบบแบ่งปันทรัพยากรที่ใช้ไม่ได้ให้สิทธิ์รันคำสั่งติดตั้งบนเซิร์ฟเวอร์

แต่หลักการที่เฟรมเวิร์กมีให้ ระบบนี้ก็ทำเองครบได้แก่ ตัวรับคำขอจุดเดียว การแยกชั้นของโค้ด และการใช้คำสั่งเตรียมไว้ล่วงหน้า

16. ทำไมไม่ตั้ง cron job สำหรับปิดการเข้าใช้ที่ค้างอยู่

โฮสต์ที่ใช้จำกัดจำนวนงานตามเวลา ระบบจึงตรวจงานค้างแตรกอยู่ในทุกคำขอที่มีคนเปิดเว็บ ต้นทุนของการตรวจนี้คือคำสั่ง SQL เพียงคำสั่งเดียวที่มีดัชนีรองรับ ซึ่งต่ำมากเมื่อเทียบกับปริมาณการใช้งานระดับวิทยาลัยหนึ่งแห่ง

ข้อเสียที่ยอมรับคือ ถ้าไม่มีใครเปิดเว็บเลยทั้งวัน งานค้างจะยังไม่ถูกปิดจนกว่าจะมีคนเปิด แต่เนื่องจากผลลัพธ์คือเวลาที่บันทึก ไม่ใช่เวลาที่รัน ข้อมูลที่ได้จึงยังถูกต้องเสมอ

17. ทำไมปุ่มเช็คอินกับเช็คเอาท์เป็นปุ่มเดียวกัน

เพราะสถานะของผู้ใช้มีแค่สองอย่างคือ อยู่ในห้องสมุด หรือไม่ได้อยู่ ระบบดูแลเหตุการณ์ล่าสุดของผู้ใช้คนนั้นแล้วรู้เองว่าการกดครั้งต่อไปหมายถึงอะไร การมีสองปุ่มจะเปิดโอกาสให้กดผิดปุ่ม เช่น กดเช็คอินซ้ำทั้งที่อยู่ในห้องสมุดอยู่แล้ว

สีของปุ่มเป็นตัวบอกทิศทาง สีเขียวคือกดแล้วเข้า สีแดงคือกดแล้วออก ผู้ใช้จึงไม่ต้องอ่านตัวหนังสือก็รู้ว่าการกดจะพาไปทางไหน

18. ถ้านักศึกษากดเช็คอินรัวๆ จะเกิดอะไรขึ้น

ระบบกำหนดเวลาขั้นต่ำระหว่างการกดสองครั้งของคนเดียวกันไว้ที่ 10 วินาที ถ้ากดถี่กว่านั้นจะได้รหัส 429 พร้อมข้อความบอกว่าเหลืออีกกี่วินาที และไม่มีแถวใหม่ถูกบันทึกลงฐานข้อมูล

ด่านนี้อยู่ที่ฝั่งเซิร์ฟเวอร์ ไม่ใช่แค่ปิดปุ่มในหน้าเว็บ เพราะ `/checkin` เป็น API ที่ยิงตรงได้ และการปิดปุ่มก็หายไปทันทีที่รีเฟรชหน้า ส่วนหน้าเว็บก็ปิดปุ่มพร้อมนับถอยหลังให้เห็นด้วย เพื่อให้ผู้ใช้เข้าใจก่อนกด ไม่ใช่ถูกปฏิเสธหลังกด

19. ระบบรู้ได้ยังไงว่าใครยังอยู่ในห้องสมุด

ดูจากเหตุการณ์ล่าสุดของแต่ละคน ถ้าแถวล่าสุดเป็น `in` แปลว่ายังอยู่ ถ้าเป็น `out` แปลว่าออกไปแล้ว คำสั่งที่ใช้คือหาเลข `log_id` สูงสุดของแต่ละผู้ใช้ แล้วดูว่าแถวนั้นเป็นชนิดอะไร
ที่ใช้ `log_id` ไม่ใช่ `timestamp` เพราะถ้ามีสองแถวที่เวลาตรงกันพอดี `log_id` ยังบอกลำดับได้แน่นอน แต่เวลาบอกไม่ได้

20. ทำไมห้องสมุดเปิดแค่จันทร์ถึงศุกร์ แล้วบล็อกยังงี้

เป็นวันทำการจริงของห้องสมุดครับ ระบบเก็บเป็นค่าตั้งค่าชื่อ `LIBRARY_OPEN_DAYS` ที่แก้ได้โดยไม่ต้องแก้โค้ด เพื่อภาคเรียนไหนเปิดเสาร์

สิ่งที่ถูกบล็อกคือการเช็คอินเท่านั้น ไม่ใช่การเข้าสู่ระบบ เพราะนักศึกษายังต้องเปิดดูประวัติของตัวเองได้ และเจ้าหน้าที่ยังต้องทำรายงานย้อนหลังในวันหยุดได้ ส่วนการเช็คเอาต์ยังทำได้เสมอ เพื่อกรณีที่มีคนค้างสถานะอยู่ในห้องสมุดข้ามวันมา จะได้ไม่ติดค้างตลอดสัปดาห์

กลุ่มที่ 4 — การนำขึ้นใช้งานและการดูแล

21. Hostinger คืออะไร ใช้ทำอะไร

เป็นผู้ให้บริการพื้นที่เว็บแบบแบ่งปันทรัพยากร คือมีเครื่องเซิร์ฟเวอร์หนึ่งเครื่องที่หลายเว็บใช้ร่วมกันโดยกันพื้นที่ของแต่ละเว็บไว้ สิ่งที่ได้มาคือ PHP และ MariaDB ที่ติดตั้งไว้ให้แล้ว ชื่อโดเมน `ntclibrary.com` ใ้รับรอง HTTPS บัญชี FTP สำหรับอัปไฟล์ และหน้าจัดการฐานข้อมูล

เหตุผลที่เลือกแบบนี้แทนการเช่าเซิร์ฟเวอร์เอง คือค่าบริการเหมาะกับโครงการนักศึกษา และไม่ต้องดูแลระบบปฏิบัติการเอง

22. Git ใช้ทำอะไรบ้าง ต่างจาก GitHub ยังไง

Git คือโปรแกรมเก็บประวัติการแก้ไขไฟล์ที่ทำงานอยู่ในเครื่อง ส่วน GitHub คือบริการที่เก็บสำเนาของ Git ไว้บนอินเทอร์เน็ต

ในโครงการนี้ใช้ทำ 5 อย่าง คือ หนึ่ง เก็บประวัติทุกการแก้ไขพร้อมข้อความอธิบายว่าทำอะไรและทำไม สอง ย้อนกลับได้เมื่อแก้แล้วพัง สาม เป็นสำเนาสำรองนอกเครื่อง สี่ เป็นต้นทางของระบบนำขึ้นเซิร์ฟเวอร์อัตโนมัติ และห้า กันไฟล์ที่ไม่ควรขึ้นไปด้วยไฟล์

`.gitignore`

23. อัปไฟล์ขึ้นเซิร์ฟเวอร์ยังไง

ไม่ได้ลากไฟล์ขึ้นเองครับ ใช้ GitHub Actions เป็นตัวอัปให้ตามลำดับที่กำหนดไว้ โดยต้องซั่มก่อนหนึ่งรอบเสมอ รอบซั่มจะแสดงรายชื่อไฟล์ที่จะถูกอัปโดยยังไม่เขียนอะไรจริง เมื่อตรวจว่ารายชื่อครบแล้วจึงส่งอัปจริง

หลังอัปเสร็จ ระบบจะตรวจให้อัตโนมัติสองอย่าง คือเปิด `/login` ต้องได้รหัส 200 แปลว่าเว็บยังทำงาน และเปิด `/src/auth.php` ต้องได้รหัส 404 แปลว่าไฟล์อยู่ถูกชั้น

ที่ต้องกำหนดลำดับเพราะไฟล์ในโฟลเดอร์ `src` กับ `public` พึ่งพากัน ถ้าอัปพร้อมกันทั้งหมดจะมีช่วงเวลาที่เว็บใช้งานไม่ได้

24. ถ้าอัปแล้วเว็บพัง ทำยังไง

ย้อนกลับไป commit ก่อนหน้าใน Git แล้วส่งอัปใหม่ ซึ่งใช้เวลาไม่ถึงสองนาที เพราะทุกเวอร์ชันของโค้ดถูกเก็บไว้หมด ส่วนข้อมูลในฐานข้อมูลไม่ได้รับผลกระทบจากการอัปโค้ด เนื่องจากการอัปเดตแต่ละครั้งเปลี่ยนแค่ไฟล์โปรแกรม

25. ทดสอบยังไ้ว่าระบบทำงานถูก

มีชุดทดสอบอัตโนมัติ 27 กรณีที่ทดสอบตั้งแต่สมัครสมาชิกจนถึงเปิดรายงานทุกแบบ โดยทดสอบกับฐานข้อมูลจริงไม่ใช่ข้อมูลจำลอง และชุดทดสอบสร้างบัญชีชั่วคราวของตัวเองแล้วลบทิ้งทุกครั้งเมื่อจบ

นอกจากนี้ยังตรวจไ้ยากรณ์ทุกไฟล์ด้วย `php -l` และ `node --check` และตรวจด้วยสายตาที่ความกว้างหน้าจอ 390 พิกเซล ซึ่งเป็นขนาดโทรศัพท์ทั่วไป

กลุ่มที่ 5 — คำถามเชิงลึกที่อาจถูกถาม

26. ถ้ามีคนสองคนกดเซ็คอินพร้อมกันเป๊ะ จะเกิดอะไรขึ้น

ไม่มีปัญหาครับ เพราะแต่ละคนเขียนคนละแถวและ `log_id` ที่ฐานข้อมูลสร้างให้จะไม่ซ้ำกันแน่นอน ส่วนกรณีที่คนเดียวกันส่งคำขอสองครั้งพร้อมกัน ด่านกันกดรั้ว 10 วินาทีจะทำให้คำขอที่สองถูกปฏิเสธ

27. ทำไมต้องตั้งเขตเวลาในหลายที่

เพราะมีนาฬิกาสองเรือนที่ต้องตรงกัน คือนาฬิกาของ PHP และนาฬิกาของฐานข้อมูล ระบบตั้งของ PHP ไว้ที่เวลาประเทศไทย และสั่งฐานข้อมูลให้ใช้ค่าชดเชย `+07:00` ทุกครั้งที่เชื่อมต่อ

นี่เป็นปัญหาที่เจอจริงตอนนำขึ้นเซิร์ฟเวอร์ เครื่องที่ใช้พัฒนาตั้งเป็นเวลาไทยอยู่แล้วจึงไม่เห็นปัญหา แต่ฐานข้อมูลบนเซิร์ฟเวอร์จริงใช้เวลามาตรฐานสากล ทำให้เวลาที่บันทึกทุกรายการเพี้ยนไป 7 ชั่วโมงโดยไม่มีข้อผิดพลาดใดแจ้งเตือน

และที่ใช้ค่าชดเชยเป็นตัวเลขแทนชื่อเขตเวลา เพราะการใช้ชื่อต้องอาศัยตารางเขตเวลาที่โฮสต์แบบแบ่งปันทรัพยากรมักไม่ได้ติดตั้งไว้

28. ทำไมกราฟใน PDF ถึงเป็นรูปภาพ ไม่ใช่กราฟจริง

เพราะไลบรารีสร้าง PDF ที่ใช้มีตัวแปล CSS เป็นของตัวเองซึ่งคำนวณความสูงแบบเปอร์เซ็นต์ผิด ทำให้ได้ไฟล์ที่มีหน้าว่างพ่วงมา 50 ถึง 300 หน้า

จึงเปลี่ยนวิธีเพื่อให้ไลบรารี GD วาดกราฟเป็นรูปภาพที่เสร็จสมบูรณ์ก่อน แล้วตัวสร้าง PDF มีหน้าที่เพียงวางรูปลงไป ซึ่งเป็นงานที่มันทำได้แน่นอน ข้อดีที่ตามมาคือกราฟใน PDF จะหน้าตาเหมือนกันทุกเครื่องที่เปิด

29. ระบบรองรับผู้ใช้ได้กี่คน

ปัจจุบันมีนักศึกษาในระบบมากกว่า 1,200 คน คอขวดที่แท้จริงไม่ใช่จำนวนผู้ใช้ แต่เป็นจำนวนแถวในตารางประวัติ ซึ่งได้เตรียมรับไว้แล้วด้วยดัชนีบนคอลัมน์เวลาและดัชนีผสมของผู้ใช้กับเวลา ทำให้คำสั่งที่ใช้บ่อยที่สุดไม่ต้องไล่อ่านทั้งตาราง

ในระดับวิทยาลัยหนึ่งแห่ง ข้อมูลจะเพิ่มขึ้นประมาณหลักหมื่นแถวต่อปีการศึกษา ซึ่งอยู่ในระดับที่ MariaDB จัดการได้สบาย

30. อะไรคือสิ่งที่ยังทำได้ไม่ดีพอ และจะพัฒนาต่ออย่างไร

มี 3 เรื่องครับ

หนึ่ง ยังไม่มีโทเคน CSRF แยกต่อคำขอ ปัจจุบันอาศัยการตั้งคูกก็เป็นที่ `SameSite=Lax` ซึ่งทำให้เบราว์เซอร์ไม่แนบคูกก็ไปกับคำขอที่เว็บอื่นเป็นคนส่ง แผนคือเพิ่มโทเคนต่อคำขอ

สอง การนำไฟล์ขึ้นเซิร์ฟเวอร์ใช้ FTP ซึ่งไม่เข้ารหัสระหว่างทาง แผนคือเปลี่ยนไปใช้ FTPS

สาม นโยบายความปลอดภัยของเนื้อหายังอนุญาตสคริปต์ในบรรทัด เพราะหน้าเว็บบางหน้ามีสคริปต์ที่ต้องแทรกค่าจาก PHP แผนคือเปลี่ยนไปใช้ค่าส่งต่อคำขอ

ส่วนฟีเจอร์ที่อยากทำต่อคือการเชื่อมกับระบบทะเบียนของวิทยาลัย เพื่อไม่ต้องนำเข้ารายชื่อเอง และการแจ้งเตือนผ่านแอปพลิเคชันแชทเมื่อใกล้ถึงเวลาที่ตั้งไว้

ภาคผนวก ก อภิธานศัพท์

คำศัพท์	ความหมายอย่างง่าย
API	ช่องทางให้โปรแกรมคุยกับโปรแกรม ในระบบนี้คือที่อยู่ หน้าเว็บเรียกไปเพื่อขอหรือส่งข้อมูล เช่น <code>POST</code> <code>/checkin</code>
bcrypt	วิธีแปลงรหัสผ่านให้เป็นข้อความที่แปลงกลับไม่ได้ ออกแบบ ให้คำนวณช้าโดยตั้งใจเพื่อให้เดายาก
Commit	การบันทึกหนึ่งครั้งใน Git พร้อมข้อความอธิบายว่าทำอะไร และทำไม
CSRF	การที่เว็บอื่นหลอกให้เบราว์เซอร์ของผู้ที่เข้าสู่ระบบอยู่ส่ง คำขอมาระบบเราโดยเจ้าตัวไม่รู้
ENUM	ชนิดข้อมูลที่กำหนดค่าที่เป็นไปได้ไว้ล่วงหน้า ค่าอื่นฐาน ข้อมูลจะไม่ยอมรับ
Foreign key (คีย์ นอก)	คอลัมน์ที่ชี้ไปหาแถวในอีกตาราง ฐานข้อมูลจะบังคับว่าค่าที่ ใส่ต้องมีอยู่จริงในตารางปลายทาง
FTP	วิธีส่งไฟล์ขึ้นเซิร์ฟเวอร์แบบหนึ่ง
Hash	การแปลงข้อมูลให้เป็นค่าความยาวคงที่ที่แปลงกลับไม่ได้ ต่างจากการเข้ารหัสที่ถอดกลับได้
HTTPS	การเข้ารหัสข้อมูลระหว่างเบราว์เซอร์กับเซิร์ฟเวอร์ ทำให้คน ดักกลางอ่านไม่ออก
Index (ดัชนี)	โครงสร้างที่ช่วยให้ฐานข้อมูลค้นเจอเร็วขึ้น คล้ายสารบัญ ท้ายเล่มหนังสือ
JSON	รูปแบบข้อความสำหรับส่งข้อมูลระหว่างเซิร์ฟเวอร์กับหน้า เว็บ
PDO	ตัวเชื่อมของ PHP สำหรับติดต่อฐานข้อมูล รองรับคำสั่งแบบ เตรียมไว้ล่วงหน้า
Prepared statement	คำสั่ง SQL ที่ส่งโครงคำสั่งไปให้ฐานข้อมูลแปลก่อน แล้วส่ง ค่าตามไปที่หลัง เป็นวิธีป้องกัน SQL Injection

คำศัพท์	ความหมายอย่างง่าย
Primary key (คีย์หลัก)	คอลัมน์ที่ใช้ระบุแถวแต่ละแถวได้ไม่ซ้ำกัน
Salt (เกลือ)	ค่าสุ่มที่ผสมลงไปก่อนแปลงรหัสผ่าน ทำให้รหัสผ่านที่เหมือนกันได้ผลลัพธ์ต่างกัน
Session (เซสชัน)	ข้อมูลที่เซิร์ฟเวอร์จำไว้ว่าเบราว์เซอร์นี้คือใคร ทำให้ไม่ต้องเข้าสู่ระบบใหม่ทุกหน้า
SQL Injection	การพิมพ์คำสั่ง SQL ปนเข้ามาในช่องกรอกข้อมูล เพื่อให้คำสั่งนั้นถูกนำไปทำงานจริง
XSS	การฝากโค้ด JavaScript ไว้ในข้อมูล เพื่อให้ทำงานบนหน้าจอของผู้ใช้คนอื่น
รหัสสถานะ 200 / 401 / 403 / 404 / 429	สำเร็จ / ยังไม่เข้าสู่ระบบ / เข้าสู่ระบบแล้วแต่ไม่มีสิทธิ์ / ไม่พบ / ส่งคำขอเกินไป